

Compositional Optimization Systems

A Textbook on Category-Theoretic, Probabilistic, and Plugin-Based Architectures
for RAG and Beyond

Architecture study derived from ragkit, ragopt, RAG-TTC, GEC, and TTC Garden

August 2026

Contents

Preface	9
Learning goals	11
Chapter 1. From Optimization Loops to Categories of Processes	12
1.1 Why optimization needs a theory of composition	12
Motivation	12
Definition 1.1 — Category	12
Worked example 1.1 — A retrieval pipeline	13
Why associativity matters in software	13
Counterexample 1.1 — Untyped callback composition	14
Exercises 1.1	14
1.2 Composition in parallel: why ordinary categories are not enough	14
Motivation	14
Definition 1.2 — Monoidal category	14
Worked example 1.2 — Lexical and vector retrieval	15
Definition 1.3 — Cartesian category	16
Counterexample 1.2 — “Parallel” as a boolean flag	16
Exercises 1.2	16
1.3 String diagrams: syntax you can reason about visually	17
Motivation	17
Definition 1.4 — String diagram	17
Worked example 1.3 — A cost interpreter follows the diagram	17
Counterexample 1.3 — A graph without algebra	17
Exercises 1.3	17
1.4 Free categories: separating a plan from what a plan means	18
Motivation	18
Definition 1.5 — Free category on a graph	18
Why freeness matters architecturally	18
Definition 1.6 — Functor, operational reading	18
Worked example 1.4 — Three meanings of the same plan	19
Worked API 1.1 — A minimal plan algebra	19
Counterexample 1.4 — Storing only an executor closure	20
Exercises 1.4	20
1.5 Products, copying, and the transition toward probability	20
Motivation	20
Definition 1.7 — Deterministic versus stochastic morphism	21
Exercises 1.5	21
1.6 Parameterized morphisms: an optimizer studies a family of programs	21
Motivation	21
Definition 1.8 — Parameterized morphism	21
Worked example 1.5 — Composing chunking and retrieval parameters	22
Worked API 1.2 — Parameterized maps	23
Counterexample 1.5 — The global hyperparameter map	23

Exercises 1.6	23
1.7 Optimization fields	24
Motivation	24
Definition 1.9 — Optimization field	24
Worked example 1.6 — RAG field	24
Definition 1.10 — Proposal strategy	24
Counterexample 1.6 — “The optimizer owns everything”	25
Exercises 1.7	25
1.8 Chapter synthesis: the first architectural kernel	25
Chapter 1 review problems	25
Chapter 2. Parameters, Optics, Plugins, and Multiple Semantics	27
2.1 Why “changing a parameter” deserves its own abstraction	27
Motivation	27
2.2 Lenses: lawful local access and update	27
Definition 2.1 — Lens	27
Worked example 2.1 — A lens for vector weight	28
Why validation complicates the simple lens laws	29
Worked example 2.2 — Composing lenses	29
Counterexample 2.1 — A non-lawful “setter”	29
Exercises 2.1	29
2.3 From lenses to optics	30
Motivation	30
Definition 2.2 — Optic, informal	30
Side topic — The optic coend	30
Worked example 2.3 — A backend prism	30
Counterexample 2.2 — One universal reflection-based setter	31
2.4 Interventions are richer than values	31
Motivation	31
Definition 2.3 — Intervention	31
Worked example 2.4 — ANN efSearch	32
Worked example 2.5 — Chunking	32
2.5 Plugins as algebraic signatures	32
Motivation	32
Definition 2.4 — Algebraic signature	32
Worked API 2.1 — Plugin contract	33
Definition 2.5 — Operation descriptor	34
Why plugins should declare laws	34
Counterexample 2.3 — The god-object plugin	34
Exercises 2.2	35
2.6 Typed envelopes and the boundary between static and dynamic typing	35
Motivation	35
Definition 2.6 — Typed envelope	35
Worked example 2.6 — Crossing a plugin boundary	35
Counterexample 2.4 — JSON without schema identity	35
2.7 Canonical identity and content-addressed semantics	36
Motivation	36
Definition 2.7 — Canonical encoding	36
Definition 2.8 — Semantic versus execution identity	36
Worked example 2.7 — Plan ID	36
Counterexample 2.5 — Path as identity	36
2.8 Multiple semantics by folding the free plan	37
Motivation	37
Definition 2.9 — Plan algebra	37
Worked example 2.8 — Effect analysis	37
Worked example 2.9 — Trust-boundary analysis	37
Worked example 2.10 — Provenance interpretation	37

Worked example 2.11 — Documentation as an interpreter	38
Counterexample 2.6 — Every interpreter re-parses arbitrary code	38
2.9 Effect systems: annotations versus semantics	38
Motivation	38
Definition 2.10 — Effect annotation	38
Side topic — Why not make the whole architecture “a monad”?	38
Side topic — Algebraic effects	38
2.10 Dependency semantics and invalidation	39
Motivation	39
Definition 2.11 — Dependency graph	39
Worked example 2.12 — Chunker invalidation	39
Why a dependency graph is not enough by itself	39
Counterexample 2.7 — “Rebuild all” as correctness strategy	40
2.11 A concrete plugin: toy RAG as a grafted domain	40
Worked example 2.13 — Registering a primitive	40
Worked example 2.14 — The build and query plans	41
2.12 Plugin law testing	41
Example 2.15 — Lens laws	41
Example 2.16 — Operation determinism	41
Example 2.17 — Plan typing	41
Exercise 2.3 — Design a law suite	41
2.13 Counterexample clinic: three extensibility traps	42
Trap A — Callbacks without descriptors	42
Trap B — Interfaces that mirror one implementation	42
Trap C — A plugin owns the campaign loop	42
2.14 Chapter synthesis: the small semantic kernel	42
Chapter 2 review problems	43
Chapter 3. Probability, Experiments, Decisions, and Feedback	44
3.1 Why deterministic semantics are not enough	44
Motivation	44
3.2 Markov kernels	44
Definition 3.1 — Probability distribution	44
Definition 3.2 — Markov kernel	44
Worked example 3.1 — A stochastic answer evaluator	45
Definition 3.3 — Kleisli composition for probability	45
Worked example 3.2 — Retrieval followed by stochastic generation	46
Counterexample 3.1 — Averaging away the failure state	46
Exercises 3.1	46
3.3 Markov categories: probability plus compositional wiring	46
Motivation	46
Definition 3.4 — Markov category, operational version	46
Worked example 3.3 — One sample versus two samples	47
Worked example 3.4 — Randomness as an explicit input	47
3.4 Couplings: the mathematics of paired experiments	48
Motivation	48
Definition 3.5 — Coupling	48
Worked example 3.5 — Shared seeds as a coupling	48
Definition 3.6 — Paired difference	48
Worked example 3.6 — Why pairing can reduce variance	49
Counterexample 3.2 — Invalid sharing of randomness	49
Exercises 3.2	49
3.5 Deterministic seed splitting	49
Motivation	49
Definition 3.7 — Splittable seed discipline	49
Worked API 3.1	50
Counterexample 3.3 — Global RNG	50

3.6 Metrics are observations, not decisions	50
Motivation	50
Definition 3.8 — Metric definition	50
Worked example 3.7 — RAG trial metrics	51
Definition 3.9 — Preorder and partial order	51
Definition 3.10 — Pareto dominance	51
Counterexample 3.4 — Universal weighted score	51
3.7 Ordered gates and decision algebras	51
Motivation	51
Definition 3.11 — Gate	52
Worked example 3.8 — RAG promotion sequence	52
Definition 3.12 — Lexicographic preorder	52
Worked API 3.2	52
Counterexample 3.5 — Decision logic inside the evaluator	53
3.8 Statistical claims and uncertainty	53
Motivation	53
Definition 3.13 — Estimand	53
Worked example 3.9 — Paired bootstrap	53
Definition 3.14 — Noninferiority	53
Counterexample 3.6 — Repeated peeking	54
3.9 Comparison of statistical experiments	54
Motivation	54
Definition 3.15 — Garbling, informal	54
Worked example 3.10 — Native artifact versus projected score	54
Exercises 3.3	54
3.10 Campaigns as state machines	55
Motivation	55
Definition 3.16 — Transition system	55
Definition 3.17 — Endofunctor	55
Definition 3.18 — Coalgebra	55
Worked example 3.11 — Campaign state	56
Why an append-only log matters	56
Definition 3.19 — Resumability law	56
Counterexample 3.7 — “Retry by deleting the run directory”	57
3.11 Campaigns as cybernetic systems	57
Motivation	57
Definition 3.20 — Feedback optimizer, schematic	57
Worked example 3.12 — LLM self-optimization without semantic privilege	57
Counterexample 3.8 — Self-judging self-optimization	57
3.12 Open games and decision boundaries	58
Motivation	58
Worked example 3.13 — Separate eligibility from choice	58
3.13 Worked end-to-end example: quadratic optimization plugin	58
System	58
Parameter space	58
Evaluation kernel	59
Decision policy	59
What this example teaches	59
Exercise 3.4	59
3.14 Worked RAG example: stochastic answer quality	59
Definition 3.21 — Exact pairing relation	60
Counterexample 3.9 — Pair after dropping failures	60
3.15 Simpson’s paradox and protected strata	60
Motivation	60
Worked example 3.14	60
Definition 3.22 — Protected stratum	60
Exercises 3.5	61

3.16 Campaign provenance and evidence custody	61
Motivation	61
Definition 3.23 — Evidence graph	61
Worked example 3.15 — Reproducing an LLM proposal	61
3.17 Chapter synthesis: an optimizer is an experimental state machine	61
Chapter 3 review problems	62
Chapter 4. A Composable Optimization Architecture for RAG	63
4.1 Why RAG is a good stress test for the theory	63
Motivation	63
4.2 A denotation for a RAG release	63
Definition 4.1 — RAG release	63
Why “release” is larger than “index”	64
4.3 RAG indexing as compositional derivation	64
Motivation	64
Definition 4.2 — Derivation stage	65
Worked example 4.1 — Chunking	65
Worked example 4.2 — Generated representations	65
Definition 4.3 — Derivation dependency	65
Worked example 4.3 — Chunking versus fusion	66
4.4 Querying as a typed retrieval algebra	66
Motivation	66
Definition 4.4 — Candidate	66
Definition 4.5 — Evidence	66
Definition 4.6 — Retrieval plan	66
Worked example 4.4 — Hybrid retrieval	67
Worked example 4.5 — Weighted reciprocal-rank fusion	67
Counterexample 4.1 — Hidden hydration inside reranking	67
4.5 Open ports and plugin interfaces	67
Motivation	67
Definition 4.7 — Port	68
Worked example 4.6 — Open evaluator	68
Worked example 4.7 — Open artifact store	68
4.6 The plugin family for a production RAG field	68
4.6.1 Corpus/source plugin	68
4.6.2 Chunking plugin	68
4.6.3 Representation plugin	69
4.6.4 Embedding plugin	69
4.6.5 Index backend plugin	69
4.6.6 Fusion/reranking plugin	69
4.6.7 Evidence/admission plugin	69
4.6.8 Answer/agent plugin	70
4.6.9 Evaluator plugin	70
4.7 ragkit and ragopt: separating domain semantics from campaign custody	70
Motivation	70
Definition 4.8 — Domain layer versus experiment layer	70
Proposed package shape	71
Counterexample 4.2 — ragopt knows every RAG parameter	72
4.8 Candidate spaces as plugin interfaces	72
Motivation	72
Definition 4.9 — Candidate space	72
Worked example 4.8 — A typed RAG search space	72
Side topic — Conditional spaces	72
4.9 Evaluation fidelity is determined by the intervention	73
Motivation	73
Definition 4.10 — Fidelity	73
Worked example 4.9 — Fusion weight	73

Worked example 4.10 — Chunker	73
Worked example 4.11 — Agent tool description	73
Counterexample 4.3 — One benchmark for every parameter	73
4.10 Native artifacts and projections	74
Motivation	74
Definition 4.11 — Native trial artifact	74
Definition 4.12 — Metric projection	74
4.11 Worked campaign A: fusion-weight optimization	74
Step 1 — Baseline	74
Step 2 — Space and lens	74
Step 3 — Impact closure	75
Step 4 — Trial coordinates	75
Step 5 — Retrieval evidence	75
Step 6 — Gates	75
Step 7 — Answer fidelity	75
Step 8 — Decision	75
What the core knew	75
4.12 Worked campaign B: chunking optimization	76
Step 1 — Intervention	76
Step 2 — Impact	76
Step 3 — Materialization	76
Step 4 — Label projection	76
Step 5 — Refresh amplification metric	76
Step 6 — Decision	77
4.13 Worked campaign C: exact versus ANN index	77
Motivation	77
Candidate parameters	77
Required evidence	77
Why the intervention optic matters	77
4.14 Optimization and production release semantics	77
Motivation	77
Definition 4.13 — Promotion artifact	78
Why activation is outside the optimizer	78
4.15 Plugin design rules for a small but strong backbone	78
Rule 1 — The core owns composition, not domain vocabulary	78
Rule 2 — Plugins expose generators, not arbitrary engine callbacks	78
Rule 3 — Plugins own semantic validation	78
Rule 4 — Effects are declared	78
Rule 5 — Interventions are lawful and typed	78
Rule 6 — Native evidence is retained	78
Rule 7 — Failure is evidence	79
Rule 8 — Decision policy is versioned and separate	79
Rule 9 — Optimizers are replaceable controllers	79
Rule 10 — Every semantic claim should point to a law or test	79
4.16 A self-contained Go skeleton	79
Core schemas and operations	79
Plans	80
Optics/interventions	80
Evaluation	80
Campaign	80
4.17 Interpreters for a RAG plan	81
Execution interpreter	81
Dependency interpreter	81
Cost interpreter	81
Disclosure interpreter	81
Provenance interpreter	81
Evaluation-fidelity interpreter	81

Documentation interpreter	81
4.18 Verification strategy	81
Motivation	81
4.18.1 Core algebra tests	82
4.18.2 Optic tests	82
4.18.3 Plugin conformance	82
4.18.4 Experiment tests	82
4.18.5 RAG-specific properties	82
Worked example 4.12 — Property test for total ranking	82
4.19 When the category-theoretic architecture is overkill	83
Counterexample 4.4 — One pure scalar function	83
Counterexample 4.5 — One application with no extension boundary	83
Counterexample 4.6 — False abstraction before two domains exist	83
4.20 Migration path from pragmatic RAG code	83
Stage 0 — Behavioral fixtures	83
Stage 1 — Semantic IDs and native artifacts	83
Stage 2 — Lawful intervention APIs	83
Stage 3 — Operation descriptors	84
Stage 4 — Free plans for one path	84
Stage 5 — Campaign custody	84
Stage 6 — Additional interpreters	84
Stage 7 — Pluggable proposers	84
4.21 Capstone worked example: optimizing an entire RAG release	84
Baseline release	84
Intervention family A — Fusion	84
Intervention family B — Chunking	85
Intervention family C — ANN backend	85
Campaign strategy	85
Evidence graph	85
Promotion policy	85
4.22 Chapter synthesis	86
Chapter 4 review problems	86
Appendix A. Selected exercise solutions and hints	87
A.1 Category and composition	87
A.2 Lens laws	87
A.3 Couplings	88
A.4 Cost interpreter	88
Appendix B. Reference API sketch	89
B.1 Core	89
B.2 Plugin registry	89
B.3 Free plans	90
B.4 Optics	90
B.5 Probability and seeds	91
B.6 Experiment	91
B.7 Campaign	91
Appendix C. Mathematical perspective and further reading	93
C.1 Parameterized maps and learning	93
C.2 Optics	93
C.3 Markov categories	93
C.4 Cybernetics and feedback	93
C.5 Coalgebra	94
C.6 Free syntax and interpreters	94
Appendix D. Bibliography	95

Preface

Optimization software is often introduced with an innocent-looking loop:

```
for candidate in candidates:
    score = evaluate(candidate)
return best(candidate)
```

That loop is not wrong. It is merely the special case in which almost every difficult question has already been answered. The candidates are assumed to live in one flat space. Changing one candidate is assumed not to require rebuilding a graph of dependent artifacts. Evaluation is assumed to have one result, not a structured trace of successes, failures, costs, safety properties, and stochastic outcomes. The evaluator is assumed to be comparable across candidates. The meaning of “best” is assumed to be a total ordering. The environment is assumed to remain fixed. None of these assumptions survives contact with a serious retrieval-augmented generation system.

Consider a production RAG release. One candidate changes the weight of vector retrieval. Another changes the chunker. Another changes the embedding model. Another changes a reranker that receives source text over a network boundary. Another changes the agent prompt that decides whether to call retrieval at all. Another changes only the retry policy. These interventions do not have the same dependency closure, the same security consequences, the same evaluation unit, or even the same notion of semantic equivalence. A flat dictionary of hyperparameters can encode all of them syntactically, but it does not *explain* any of them.

This textbook develops a mathematical and software vocabulary for that explanation. Its main object is the **optimization field**: the structured environment in which systems can be constructed, locally changed, evaluated under uncertainty, compared under explicit policies, and improved repeatedly. Category theory supplies the language of composition. Parameterized morphisms and optics supply the language of controlled intervention. Probability kernels supply the language of stochastic evaluation and paired comparison. Coalgebra and state-machine semantics supply the language of repeated feedback. Plugin interfaces supply the engineering boundary by which specific domains—RAG in particular—can be grafted onto a small core without making that core a bag of callbacks.

The book is written as four large chapters rather than as a thesis. Every major abstraction is introduced in a recurring pedagogical sequence:

1. **Motivation.** What concrete problem forces us to introduce the abstraction?
2. **Definition.** What is the mathematical or software object?
3. **Worked examples.** How does it behave in small examples and in RAG?
4. **Counterexamples.** What goes wrong if we weaken or misuse it?
5. **Exercises.** What should you be able to derive or implement yourself?

The intended reader is comfortable with programming and basic discrete mathematics. Familiarity with functions, types, interfaces, probability, and ordinary data structures is enough. Category theory is developed from composition and products upward; no prior knowledge of monoidal categories, Markov categories, lenses, or coalgebra is assumed.

The implementation examples use Go-like APIs because the motivating codebase is written primarily in Go, but the architecture is language-independent. The categorical structures are design constraints, not requirements that an application expose mathematical vocabulary in its public API.

How to read the mathematics. A formula in this book is useful when it lets us state a

software law precisely. For example, associativity is useful because it lets us regroup a pipeline without changing its meaning; a lens law is useful because it makes “change exactly this field” testable; a coupling is useful because it defines what “paired randomness” means in an experiment. When a mathematical abstraction does not create such a law, interface, or test, we do not introduce it.



Figure 1: The objects of an optimization field: domain state, compositional plans, interventions, evaluation, evidence, decisions, and feedback.

Learning goals

By the end of the book, you should be able to:

- explain why a typed optimization pipeline is naturally a category of processes;
- distinguish cartesian products from a general monoidal tensor and explain why explicit copying matters for effects and probability;
- construct a free typed process language and interpret one plan into execution, cost, dependency, provenance, and policy analyses;
- model a tunable stage as a parameterized morphism and compose parameter spaces systematically;
- define a lawful lens and use lens laws to test localized interventions;
- design plugin interfaces as algebraic signatures with laws rather than unrestricted callbacks;
- model stochastic evaluation as a Markov kernel and explain why paired experiments require an explicit coupling;
- distinguish metrics, constraints, partial orders, and decision policies;
- formulate an iterative optimizer as a coalgebraic feedback process with resumable evidence;
- map RAG indexing and querying into this backbone without making RAG concepts primitive in the core;
- compute dependency and invalidation closures for RAG interventions;
- design a practical Go package boundary in which `ragkit` supplies domain semantics, `ragopt` supplies experiment custody, and domain plugins supply operations and laws;
- recognize cases where the categorical abstraction is too strong, too weak, or simply unnecessary.

Chapter 1. From Optimization Loops to Categories of Processes

1.1 Why optimization needs a theory of composition

Motivation

Suppose we want to optimize a hybrid retrieval system. Its query path looks roughly like this:

```
query
-> rewrite
-> lexical search ----\
                        +--> fuse -> rerank -> context -> answer
-> vector search -----/
```

At first we might represent each candidate as a record:

```
type Candidate struct {
    LexicalK      int
    VectorK       int
    VectorWeight  float64
    RerankK       int
}
```

This works until we change chunking. Then the candidate is not merely a query-time parameter. It changes the chunks, which changes representations, which changes embeddings, which changes indexes, which changes every downstream evaluation. If we change only `VectorWeight`, rebuilding the index is wasteful and weakens the causal interpretation of the experiment. If we change the embedding model but reuse the old vector index, the candidate is invalid. The optimizer therefore needs to know how computations compose and which outputs depend on which inputs.

The minimal mathematical question is not “what optimizer should we use?” It is:

What is the structure of the computations being optimized, and what laws let us assemble large computations from small ones?

That is exactly the question category theory asks.

Definition 1.1 — Category

A **category** $\mathcal{C}$ consists of:

1. a collection of **objects** $A, B, C, \dots$;
2. for each pair of objects A, B , a collection of **morphisms** $f : A \rightarrow B$;
3. for composable morphisms $f : A \rightarrow B$ and $g : B \rightarrow C$, a composite

$$g \circ f : A \rightarrow C;$$

4. for every object A , an identity morphism $\text{id}_A : A \rightarrow A$;
5. the laws

$$h \circ (g \circ f) = (h \circ g) \circ f$$

and

$$\text{id}_B \circ f = f = f \circ \text{id}_A.$$

The first law is **associativity**; the second is the **identity law**.

The definition is abstract because the words “object” and “morphism” are intentionally neutral. In a programming category, objects can be types and morphisms can be total functions. In a database category, objects can be schemas and morphisms transformations. In our optimization architecture, objects are typed interfaces or schemas and morphisms are declared computational stages.

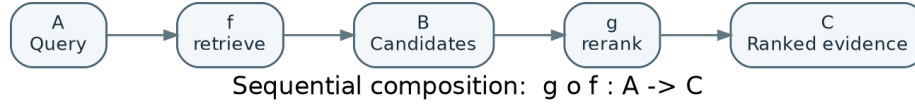


Figure 2: Sequential composition of retrieval and reranking.

Worked example 1.1 — A retrieval pipeline

Let:

- Q be the type of normalized queries;
- C be the type of candidate rankings;
- E be the type of ranked evidence.

Define

$$\text{retrieve} : Q \rightarrow C$$

and

$$\text{rerank} : C \rightarrow E.$$

Then the query-stage pipeline is simply

$$\text{rerank} \circ \text{retrieve} : Q \rightarrow E.$$

The type C is not decoration. It is the boundary that tells us whether the stages can be composed. If a new reranker expects hydrated text but retrieval produces only document IDs, then its domain is not C but some other object H . A hydration morphism

$$\text{hydrate} : C \rightarrow H$$

must appear explicitly. The correct plan becomes

$$\text{rerank} \circ \text{hydrate} \circ \text{retrieve}.$$

This simple type distinction already has a security consequence. If authorization must occur before source text leaves the machine, then the plan must contain an authorization stage before hydrate or before any remote reranker. A raw callback chain can hide this ordering; a typed composition makes it inspectable.

Why associativity matters in software

Associativity says that the following two groupings have the same meaning:

$$(h \circ g) \circ f = h \circ (g \circ f).$$

In code, this means a pipeline can be packaged into subpipelines without changing semantics. We can define

```
retrieval := Seq(rewrite, retrieve, fuse)
answering := Seq(rerank, context, generate)
full := Seq(retrieval, answering)
```

or expand all the stages in one list. If our composition primitive does not satisfy associativity, “refactoring” a plan into a reusable component may change what it does. That is a severe architecture smell.

Fundamentals: laws are refactoring contracts. Category laws are not merely elegant equations. Associativity licenses module boundaries. Identity licenses adapters that preserve meaning. Later, lens laws will license local parameter updates, and Markov-category laws will license rearrangements of stochastic processes.

Counterexample 1.1 — Untyped callback composition

Consider:

```
type Step func(context.Context, map[string]any) (map[string]any, error)
```

Every stage has the same apparent type, so every stage composes with every other stage. This looks flexible, but it makes *ill-typed plans representable*. A reranker can be connected directly to raw source documents, an evaluator can receive an index manifest instead of retrieval results, and a security filter can accidentally be placed after disclosure. All errors move from construction time to runtime.

The category-theoretic lesson is not “never use maps.” It is that a compositional kernel should represent the domain and codomain of operations strongly enough to reject meaningless compositions.

Exercises 1.1

1. Let $f : A \rightarrow B$, $g : B \rightarrow C$, and $h : C \rightarrow D$. Write the types of $g \circ f$, $h \circ g$, and $h \circ g \circ f$. Explain why the last expression is unambiguous.
2. Model a compiler pipeline `parse -> typecheck -> optimize -> codegen` as a category of typed stages. What are plausible objects between stages?
3. In a RAG system, a remote reranker expects `[]HydratedChunk`. Retrieval produces `[]HitID`. Authorization consumes `Subject × []HitID` and produces `[]AuthorizedHitID`. Draw a valid typed pipeline that guarantees authorization before hydration.
4. Construct one example of a `map[string]any` pipeline that type-checks at the host-language level but is semantically nonsensical.

1.2 Composition in parallel: why ordinary categories are not enough

Motivation

Hybrid retrieval does not only compose stages sequentially. Lexical and vector retrieval can be executed independently from the same query. Cost analysis cares about the difference: sequential work adds latency; independent work may share a critical path. Security analysis cares too: two branches may have different effects. A mathematical model that knows only sequential composition cannot express this distinction.

We therefore need a second operation: composition **side by side**.

Definition 1.2 — Monoidal category

Fundamentals: why the word “monoidal”? A **monoid** is a set equipped with an associative binary operation and an identity element. Integers under addition form a monoid:

$(a+b)+c=a+(b+c)$ and $0+a=a$. A monoidal category categorifies this pattern: the binary operation acts on objects and morphisms, and the unit is an object rather than an element. The associativity and unit laws hold through coherent structural isomorphisms rather than literal equality in the most general presentation.

A **monoidal category** is a category $\mathcal{C}$ equipped with:

- a binary operation on objects $A \otimes B$, called the **tensor product**;
- a corresponding operation on morphisms

$$f \otimes g : A \otimes C \rightarrow B \otimes D$$

whenever $f : A \rightarrow B$ and $g : C \rightarrow D$;

- a distinguished **unit object** I ;
- coherent associativity and unit isomorphisms.

A **symmetric monoidal category** additionally has a symmetry

$$\sigma_{A,B} : A \otimes B \rightarrow B \otimes A$$

satisfying coherence laws.

For software architecture, $\otimes$ means “place these interfaces or computations side by side.” It does **not** automatically mean “run them concurrently”; concurrency is one possible interpreter. The syntax records independence, and an execution interpreter may exploit it.

Worked example 1.2 — Lexical and vector retrieval

Let

$$L : Q \rightarrow R_L$$

and

$$V : Q \rightarrow R_V.$$

To feed the same query to both branches we need a copying operation

$$\Delta_Q : Q \rightarrow Q \otimes Q.$$

Then the combined channel computation is

$$(L \otimes V) \circ \Delta_Q : Q \rightarrow R_L \otimes R_V.$$

Finally a fusion operation

$$F : R_L \otimes R_V \rightarrow R$$

gives

$$F \circ (L \otimes V) \circ \Delta_Q : Q \rightarrow R.$$

The explicit copy may look pedantic when Q is an immutable Go value. It becomes crucial when we move to probability and effects. Copying a realized sample is different from sampling twice. Copying a capability token may be illegal. Copying a large artifact has different operational meaning from passing two references. A general monoidal core does not assume copying for free.



Tensor/parallel composition models independent branches; copy is explicit.

Figure 3: Hybrid retrieval as explicit copying followed by parallel channel composition.

Definition 1.3 — Cartesian category

A category is **cartesian** when the monoidal product is a categorical product. Informally, this gives canonical operations

$$\Delta_A : A \rightarrow A \times A$$

for copying and

$$!_A : A \rightarrow 1$$

for discarding.

Ordinary pure functions between immutable values are usually modeled cartesianly. But many effectful and probabilistic categories are not cartesian in the same way. This is one reason to keep a monoidal process language smaller than the host language.

Side topic: affine, relevant, and linear usage. If values may be discarded but not copied, the structure is often called affine; if copied but not discarded, relevant; if neither operation is freely available, linear. Capability systems, file handles, unique references, and stochastic samples often benefit from making these structural operations explicit.

Counterexample 1.2 — “Parallel” as a boolean flag

A workflow API sometimes represents parallelism like this:

```

Step{Name: "lexical", Parallel: true}
Step{Name: "vector", Parallel: true}
Step{Name: "fuse", Parallel: false}
  
```

The flag does not tell us *which outputs* are independent, which inputs are duplicated, or where the branches join. It describes a scheduling preference, not composition. A tensor structure expresses the data dependency first; concurrency can then be derived by an interpreter.

Exercises 1.2

1. Draw a tensor diagram for preprocessing two independent corpora and then merging their indexes.
2. Explain why $A \otimes B$ should not automatically be identified with a Go struct $\{A; B\}$ in every interpreter.
3. Suppose a stochastic operation `sample: Seed -> Result` is copied before execution versus its result copied after execution. Write the two diagrams and explain why their distributions differ.
4. Give one optimization-analysis question that needs to distinguish sequential work from independent branches.

1.3 String diagrams: syntax you can reason about visually

Motivation

Long chains of composition symbols become unreadable once copying, branching, and feedback appear. Monoidal categories admit a graphical notation called **string diagrams**. In the notation, wires are objects and boxes are morphisms. Sequential composition connects boxes left-to-right or top-to-bottom; tensor composition places them side by side.

The key point is not visual prettiness. String diagrams provide a syntax in which equations often become topological deformations. A diagram can also map directly to a plan IR.

Definition 1.4 — String diagram

A **string diagram** for a monoidal category is a graphical representation in which:

- each wire carries an object/type;
- each box has typed input and output wires;
- connecting an output to an input denotes composition;
- juxtaposition denotes tensor product;
- crossing wires denotes symmetry in a symmetric monoidal category.

For a free monoidal category generated by a set of primitive operations, well-typed string diagrams are essentially programs in a graphical language.

Worked example 1.3 — A cost interpreter follows the diagram

Consider lexical and vector retrieval in parallel, followed by fusion. Suppose estimated costs are:

$$C_L = (4 \text{ ms}, 2 \text{ work units}), \quad C_V = (9 \text{ ms}, 5 \text{ work units}), \quad C_F = (1 \text{ ms}, 1 \text{ work unit}).$$

If the first coordinate is critical-path latency and the second is total work, a tensor interpreter can combine branches as

$$C_{L \otimes V} = (\max(4, 9), 2 + 5) = (9, 7).$$

Sequential fusion then yields

$$C = (9 + 1, 7 + 1) = (10, 8).$$

The same syntax can be interpreted into actual execution, where the branches run concurrently; into resource planning, where network permits are checked; or into provenance, where their result identities are recorded separately.

Counterexample 1.3 — A graph without algebra

A DAG is not automatically a compositional semantics. It may tell us that node fuse depends on nodes lexical and vector, but without typed ports and composition laws it does not tell us how outputs line up, whether duplicating an input is meaningful, or what equivalences preserve semantics. The graph data structure becomes powerful when it is the representation of a typed algebra, not when it replaces one.

Exercises 1.3

1. Draw a string diagram for query $\rightarrow$ rewrite $\rightarrow$ (lexical $\otimes$ vector) $\rightarrow$ fuse $\rightarrow$ rerank.
2. Annotate each box with an effect class such as CPU, network, or artifact-read.
3. Define a critical-path interpretation for a diagram in which tensor takes max latency and sequence takes +. What algebraic assumptions are you making?

1.4 Free categories: separating a plan from what a plan means

Motivation

A central architecture requirement is that the system should be able to inspect an optimization plan **before executing it**. We want to ask:

- Which operations occur?
- What network effects can happen?
- Which parameters influence the result?
- What artifacts are read and written?
- What is the estimated cost?
- Is source text disclosed remotely?
- Can two plans be given stable semantic identities?

If a plan is represented as already-executable closures, much of this structure is gone. We need syntax that records composition without immediately choosing an execution semantics.

Definition 1.5 — Free category on a graph

Given a directed graph of objects and primitive arrows, the **free category** generated by that graph has:

- the same objects;
- morphisms given by finite well-typed paths of primitive arrows;
- identity morphisms given by empty paths;
- composition given by path concatenation.

The word *free* means that no equations between generated morphisms are imposed except those required by the category laws.

For a monoidal signature, one analogously forms a **free symmetric monoidal category**, whose terms include sequence, tensor, identities, and symmetries. Work on free gs-monoidal and Markov categories makes this idea concrete for string diagrams with copy and discard [Fritz and Liang, 2023].

Why freeness matters architecturally

Suppose plugins declare primitive operations such as:

```
chunk    : Spec @ Corpus -> Chunks
index    : Chunks -> Index
retrieve: Index @ Spec @ Case -> Retrieval
measure  : Retrieval -> Trial
```

The core can build a syntax tree or graph from these generators without knowing how chunk works. Because the plan is free syntax, any **interpreter** that assigns a meaning to each generator and preserves the category/monoidal operations extends to the whole plan.

This is the universal property that makes multiple interpretations principled rather than ad hoc.

Definition 1.6 — Functor, operational reading

A **functor** $F : \mathcal{C} \rightarrow \mathcal{D}$ maps objects and morphisms from one category to another while preserving identities and composition:

$$F(\text{id}_A) = \text{id}_{F(A)}, \quad F(g \circ f) = F(g) \circ F(f).$$

A monoidal functor additionally preserves tensor structure up to the required coherence.

In our design, the source category is often the free category of plans. An execution interpreter is a functor into a category of effectful computations. A cost interpreter is a functor into a simpler algebra of costs. A dependency interpreter maps each operation to the set of semantic dependencies it touches.

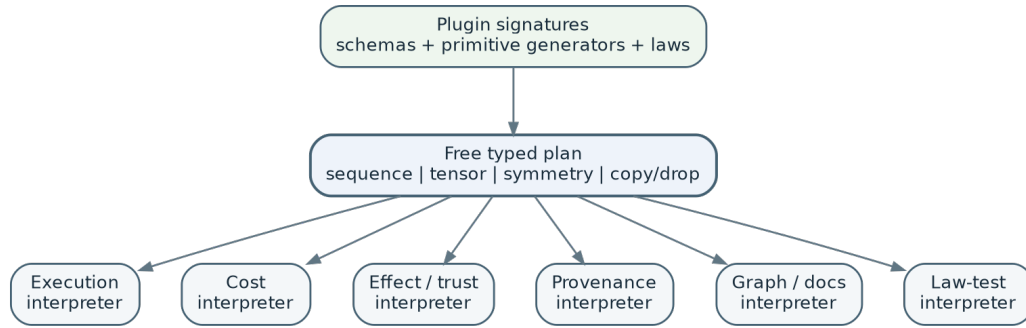


Figure 4: One free plan can be folded into execution, cost, effect, provenance, graph, and law-test interpretations.

Worked example 1.4 — Three meanings of the same plan

Let

$$p = \text{measure} \circ \text{retrieve} \circ \text{index} \circ \text{chunk}.$$

Execution interpretation. Map each generator to an actual Go implementation. The result runs the pipeline.

Dependency interpretation. Assign

```

chunk    -> {corpus, chunk-policy}
index    -> {chunks, index-backend}
retrieve -> {index, query-policy}
measure  -> {retrieval-output, metric-policy}

```

and combine dependencies by set union. The plan yields the full dependency set.

Cost interpretation. Assign each generator an estimated pair (work, criticalPath) and compose it according to the plan structure.

The plan is one value; the interpreters are separate. This is preferable to scattering execution, cost, and policy logic through every orchestration function.

Worked API 1.1 — A minimal plan algebra

```
type PlanKind uint8
```

```
const (
    Identity PlanKind = iota
    Primitive
    Sequence
    Tensor
    Permute
    Copy
    Drop
)
```

```
type Plan struct {
```

```

Kind      PlanKind
In, Out   Port
Operation OperationID
Children  []*Plan
Perm      []int
}

```

An interpreter can be expressed as a fold:

```

type Algebra[R any] interface {
  Identity(Port) (R, error)
  Primitive(OperationDescriptor) (R, error)
  Sequence(in, out Port, children []R) (R, error)
  Tensor(in, out Port, children []R) (R, error)
  Permute(in, out Port, permutation []int) (R, error)
  Copy(schema SchemaID) (R, error)
  Drop(schema SchemaID) (R, error)
}

```

```

func Fold[R any](p *Plan, sig Signature, a Algebra[R]) (R, error)

```

This is ordinary structural recursion. The category-theoretic content is in the constructors and the preservation laws expected of the interpreter.

Counterexample 1.4 — Storing only an executor closure

```

type Plan func(context.Context) error

```

This representation can run, but it cannot reliably answer what it will run. Static effect analysis becomes reflection or convention. Stable plan identity becomes difficult because closure identity is not semantic identity. Visualization requires manual duplication of structure. Testing a security property like “no remote disclosure before authorization” requires running or instrumenting the plan instead of inspecting it.

Exercises 1.4

1. Define an interpreter from a plan into the number of primitive operations.
2. Define an interpreter into the set of effect labels appearing in a plan.
3. Explain why the free plan should normally record semantic operation IDs rather than Go function pointers.
4. Suppose two plans differ only by insertion of an identity node. Should their canonical plan identity be equal? Design a normalization rule and discuss its consequences.

1.5 Products, copying, and the transition toward probability

Motivation

We now have two related structures:

- a free symmetric monoidal syntax useful for arbitrary processes;
- a host language in which ordinary values are freely copied and discarded.

It is tempting to make the plan category cartesian and be done. But probability gives a reason to resist that shortcut.

Suppose a kernel samples a random reranker response. There are two distinct operations:

1. sample once and copy the realized response;
2. execute the stochastic kernel twice independently.

If copying a stochastic morphism were indistinguishable from tensoring it with itself, the model would collapse these behaviors.

Definition 1.7 — Deterministic versus stochastic morphism

A **deterministic morphism** maps each input to exactly one output. A **stochastic morphism** maps each input to a probability distribution over outputs.

In finite form, a stochastic morphism $K : X \rightarrow Y$ is a Markov kernel assigning to every $x \in X$ a distribution $K(\cdot \mid x)$ over Y .

We develop this fully in Chapter 3. For now, the important lesson is architectural: explicit copy and discard operations make the transition from deterministic to stochastic interpretation possible without changing the plan language completely.

Design note: do not overfit the core to pure functions. A RAG optimizer will eventually interpret plans using local deterministic code, remote stochastic models, caches, artifact stores, human judgments, and stateful campaign records. The syntax should be simpler than all of these semantics, not isomorphic to the first one you implement.

Exercises 1.5

1. Let K be a fair coin kernel from the unit object to $\{H, T\}$. Compare the joint distribution of (X, X) when one sample is copied with the distribution of (X_1, X_2) when the kernel runs independently twice.
2. Which of `FileHandle`, `Digest`, `HTTPResponse`, and `RandomSample` would you be comfortable copying freely? Explain operational versus semantic copying.

1.6 Parameterized morphisms: an optimizer studies a family of programs

Motivation

A fixed retrieval function is not yet an optimization problem. Optimization begins when behavior depends on a tunable value.

For example:

$$\text{retrieve}_{w,k} : Q \rightarrow R$$

may depend on vector weight w and candidate depth k . The software system therefore needs to represent a **family of morphisms** indexed by parameters.

Definition 1.8 — Parameterized morphism

Given a monoidal category $\mathcal{C}$, a parameterized morphism from A to B with parameter object P is an ordinary morphism

$$f : P \otimes A \rightarrow B.$$

We write this suggestively as

$$f_p : A \rightarrow B.$$

The parameter is still an explicit input. The notation merely emphasizes that optimization often holds p fixed while evaluating many a 's.

The **Para construction** packages such parameterized morphisms into a category. Composition pairs parameter spaces. If

$$f : P \otimes A \rightarrow B$$

and

$$g : Q \otimes B \rightarrow C,$$

then their parameterized composite has parameter space $P \otimes Q$:

$$(Q \otimes P) \otimes A \rightarrow C,$$

up to the symmetry and associativity of the monoidal category.

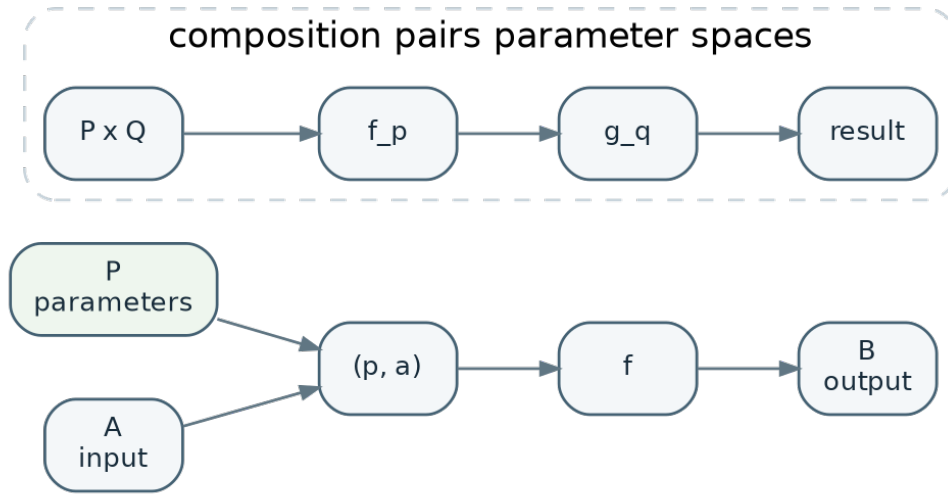


Figure 5: A parameterized morphism and the way composition accumulates parameter spaces.

Worked example 1.5 — Composing chunking and retrieval parameters

Let

$$\text{chunk} : P_C \otimes D \rightarrow C$$

and

$$\text{retrieve} : P_R \otimes (C \otimes Q) \rightarrow H.$$

The composite system depends on both parameter objects:

$$(P_C \otimes P_R) \otimes (D \otimes Q) \rightarrow H.$$

A concrete ReleaseSpec can be interpreted as an element of the composite parameter space:

```
type ReleaseSpec struct {
  Chunk ChunkSpec
  Query RetrievalSpec
}
```

The theoretical benefit is not that a struct becomes a tensor product. The benefit is that parameter accumulation is compositional: when systems are composed, their tunable spaces compose too. We do not need one global registry of string keys.

Worked API 1.2 — Parameterized maps

```
type Parametric[P, A, B any] struct {
  Run func(P, A) (B, error)
}
```

```
type Pair[A, B any] struct {
  First  A
  Second B
}
```

```
func Compose[P, Q, A, B, C any](
  f Parametric[P, A, B],
  g Parametric[Q, B, C],
) Parametric[Pair[P, Q], A, C]
A reparameterization
```

$$r : R \rightarrow P$$

turns $f : P \otimes A \rightarrow B$ into a family indexed by R :

$$f \circ (r \otimes \text{id}_A) : R \otimes A \rightarrow B.$$

This will matter when a product-level configuration compiles into the lower-level parameters of multiple plugins.

Counterexample 1.5 — The global hyperparameter map

```
map[string]any{
  "chunk.size": 800,
  "retrieval.vector_weight": 1.2,
  "reranker.model": "...",
}
```

A map can be a useful serialization format, but it is a poor semantic core. Composition is string convention. There is no type-safe way to know which subprogram consumes which key. Dependency analysis becomes hand-maintained. Two plugins can collide on names. Invalid candidates become ordinary runtime values.

The Para perspective says that each component owns a parameter object and composition pairs those objects. Serialization can be derived later.

Exercises 1.6

1. Let a chunker have parameters (size, overlap) and a retriever have parameters (lexicalK, vectorK, weight). Write the composite parameter type in a typed language.
2. What happens to the parameter space when a third reranker stage is composed?
3. Explain the difference between a parameter and a runtime input. Why is a query normally an input rather than a parameter in an optimization campaign?
4. Give an example where the same low-level parameter is intentionally shared by two stages. How might reparameterization express that sharing?

1.7 Optimization fields

Motivation

A category of parameterized computations still does not describe optimization. We need workloads, evaluators, measurements, interventions, and decisions. It is useful to name the whole structured environment so we can distinguish it from any individual search algorithm.

Definition 1.9 — Optimization field

An **optimization field** is a tuple of interacting structures

$$\mathfrak{F} = (\mathcal{C}, \mathcal{P}, \mathcal{I}, \mathcal{W}, \mathcal{E}, \mathcal{M}, \mathcal{D}),$$

where:

- $\mathcal{C}$ is a category or typed algebra of composable computations;
- $\mathcal{P}$ is the family of parameter spaces attached to computations;
- $\mathcal{I}$ is a collection of legal interventions on those parameters;
- $\mathcal{W}$ is a workload space: cases, tasks, traffic, simulations, or scenarios;
- $\mathcal{E}$ is an evaluation semantics, often stochastic;
- $\mathcal{M}$ is a measurement structure extracting comparable observations;
- $\mathcal{D}$ is a decision policy or algebra determining eligibility and preference.

A **campaign** is an execution inside an optimization field with a baseline, proposal strategy, budget, accumulated evidence, and termination rule.

This definition is intentionally broader than “hyperparameter optimization.” It includes compiler autotuning, index design, prompt optimization, retrieval policy, scheduling policy, cache strategy, and system configuration.

Worked example 1.6 — RAG field

For one retrieval-optimization campaign:

- $\mathcal{C}$: typed indexing and query plans;
- $\mathcal{P}$: chunking, representation, embedding, index, fusion, reranking, context, agent settings;
- $\mathcal{I}$: lawful changes such as “replace vector weight” or “swap chunker spec”;
- $\mathcal{W}$: labeled retrieval cases plus answer/session scenarios;
- $\mathcal{E}$: exact retrieval plus stochastic model calls;
- $\mathcal{M}$: recall, MRR, nDCG, grounding, latency, cost, disclosure events;
- $\mathcal{D}$: ordered gates—security, integrity, quality non-regression, target improvement, cost.

Notice that Bayesian optimization, grid search, evolutionary search, or an LLM proposer is not present in the definition. Those are **proposal strategies** inside the field. This separation is important: the semantics of what counts as a legal, reproducible, comparable experiment should not depend on the algorithm that proposes candidates.

Definition 1.10 — Proposal strategy

A **proposal strategy** is a rule that maps current campaign state to one or more legal interventions:

$$\pi : S \rightarrow \mathcal{I}^*.$$

The strategy may be deterministic, randomized, learned, human-directed, or LLM-driven. It does not define the field’s legality or decision policy.

Counterexample 1.6 — “The optimizer owns everything”

A monolithic optimizer that can mutate arbitrary configuration, trigger arbitrary builds, choose its own evaluator, omit failed trials, and activate production has no stable semantic boundary. It is simultaneously proposer, experimenter, judge, and deployer. Even if its search strategy is sophisticated, the resulting evidence is difficult to trust.

A composable field constrains the optimizer: it proposes through legal intervention interfaces, evaluates through fixed or versioned evaluators, records complete evidence, and submits results to an independent decision policy.

Exercises 1.7

1. Define an optimization field for compiler flag tuning. Identify each component of $\mathfrak{F}$.
2. Define one for database query-plan selection.
3. In a RAG field, where should the LLM judge live: $\mathcal{E}$, $\mathcal{M}$, $\mathcal{D}$, or the proposal strategy? Argue for a decomposition rather than a single answer.
4. Explain why grid search and Bayesian optimization are not adequate top-level domain abstractions for a reusable architecture.

1.8 Chapter synthesis: the first architectural kernel

We can now state the smallest useful core obtained from Chapter 1.

The core needs:

1. **typed objects or schemas** representing ports;
2. **primitive operation descriptors** supplied by domains;
3. **free composition** by sequence, tensor, symmetry, and explicit structural operations;
4. **plan validation** before interpretation;
5. **multiple interpreters** expressed as folds or functors;
6. **parameterized components** whose parameter spaces compose.

It does *not* yet need:

- a universal candidate struct;
- a global hyperparameter map;
- a built-in RAG type;
- a specific search algorithm;
- an LLM;
- a production scheduler;
- a universal scalar objective.

This smallness is deliberate. The next chapter asks how domain-specific changes and implementations can attach to the kernel without weakening its laws.

Chapter 1 review problems

1. Prove, informally but precisely, that flattening nested Sequence nodes preserves the denotation of a plan if the interpreter preserves category composition.
2. Design a canonical normalization for $\text{Tensor}(\text{Tensor}(f,g),h)$ and $\text{Tensor}(f,\text{Tensor}(g,h))$. What information must be preserved if execution scheduling is allowed to differ?
3. A product configuration contains one embedding model used both at build time and query time. Use reparameterization to model this as a single parameter rather than two independently tunable values.
4. Compare two designs: (a) a free plan interpreted into execution, and (b) a direct builder that emits executable closures plus a separate hand-maintained metadata graph. List failure modes in which the two representations drift.

5. Explain why an optimization field is closer to a *scientific experimental apparatus* than to a function minimizer.

Chapter 2. Parameters, Optics, Plugins, and Multiple Semantics

2.1 Why “changing a parameter” deserves its own abstraction

Motivation

Chapter 1 gave us parameterized morphisms. A system can depend on a parameter object P , and composed systems accumulate parameter spaces. But an optimizer does not merely *read* parameters. It changes them.

That apparently small step introduces a new correctness problem. Suppose a RAG release specification is:

```
type ReleaseSpec struct {  
    Chunking ChunkSpec  
    Embedding EmbeddingSpec  
    Retrieval RetrievalSpec  
    Reranker  RerankerSpec  
    Answer    AnswerSpec  
}
```

An experiment is announced as “change only the vector fusion weight.” If candidate construction is an arbitrary function

```
func mutate(any) any
```

then the claim cannot be verified. The function might also change TopK, normalize another field, rewrite a prompt, or forget a nested version tag. A diff after the fact can detect some accidental changes, but it does not give composition laws for interventions.

We want a first-class representation of *focus*: a lawful way to view one part of a larger parameter object and replace that part while preserving everything else.

That is the role of lenses and, more generally, optics.

2.2 Lenses: lawful local access and update

Definition 2.1 — Lens

For ordinary sets or typed values, a **lens** from a structure S to a focus A consists of two operations:

$$\text{get} : S \rightarrow A$$

and

$$\text{put} : S \times A \rightarrow S.$$

A well-behaved lens is expected to satisfy three familiar laws.

Get-Put. Writing back what we just read changes nothing:

$$\text{put}(s, \text{get}(s)) = s.$$

Put-Get. Reading after writing returns what was written:

$$\text{get}(\text{put}(s, a)) = a.$$

Put-Put. Two consecutive writes are equivalent to only the final write:

$$\text{put}(\text{put}(s, a), b) = \text{put}(s, b).$$

These are not the only formulations of lens lawfulness in the literature, and general optics support much richer structures [Riley, 2018]. For configuration interventions, however, these three laws give an excellent engineering interface.

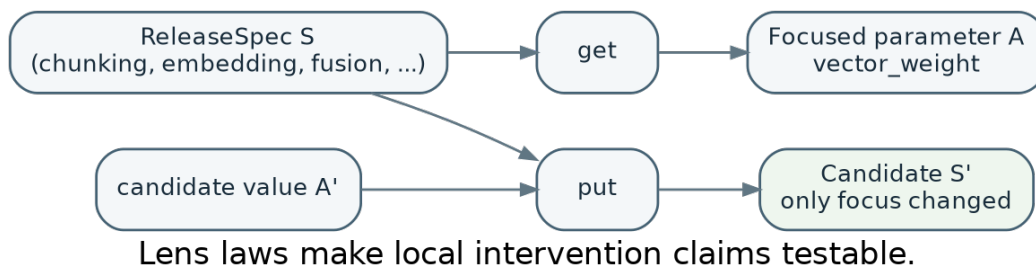


Figure 6: A lens focuses on one component of a larger release specification and reconstructs a candidate after an update.

Worked example 2.1 — A lens for vector weight

```

type RetrievalSpec struct {
    LexicalWeight float64
    VectorWeight  float64
    TopK          int
}

type ReleaseSpec struct {
    Chunking ChunkSpec
    Retrieval RetrievalSpec
}

var VectorWeight = Lens[ReleaseSpec, float64]{
    ID: "release.retrieval.vector_weight/v1",
    Get: func(s ReleaseSpec) float64 {
        return s.Retrieval.VectorWeight
    },
    Put: func(s ReleaseSpec, x float64) (ReleaseSpec, error) {
        if math.IsNaN(x) || math.IsInf(x, 0) || x < 0 {
            return s, fmt.Errorf("invalid vector weight")
        }
        s.Retrieval.VectorWeight = x
        return s, nil
    },
}
  
```

If the lens passes the three laws over a suitable generated set of valid states and values, then a campaign can make a much stronger statement than “this patch probably touched one field.” It can say that candidate materialization is performed through a tested lawful focus.

Why validation complicates the simple lens laws

Real configuration updates may reject invalid values. A chunk-overlap lens cannot accept `overlap >= chunkSize`. The put operation therefore often has type

$$S \times A \rightarrow S + \text{Error}.$$

We apply the laws to values for which put succeeds. More sophisticated treatments use partial lenses or optics in categories with effects. The software lesson is simpler: **validation is part of intervention semantics and must not be hidden after mutation.**

Worked example 2.2 — Composing lenses

Suppose we have a lens

$$L_1 : \text{ReleaseSpec} \rightsquigarrow \text{RetrievalSpec}$$

and another

$$L_2 : \text{RetrievalSpec} \rightsquigarrow \mathbb{R}.$$

Their composition focuses directly on vector weight:

$$L_2 \circ L_1 : \text{ReleaseSpec} \rightsquigarrow \mathbb{R}.$$

In code:

```
func Compose[S, A, B any](outer Lens[S,A], inner Lens[A,B]) Lens[S,B]
```

This is one of the main reasons optics fit plugin architectures. A plugin can expose lawful local focuses on its own spec. A product-level release spec can compose those focuses without the core understanding the fields.

Fundamentals: a lens is not just a getter and setter. If a setter normalizes unrelated fields, increments a hidden version, or loses information, it may still be useful application code, but it does not satisfy the laws required for a local optimization intervention. The laws are what make composition trustworthy.

Counterexample 2.1 — A non-lawful “setter”

```
func SetVectorWeight(s ReleaseSpec, x float64) ReleaseSpec {
    s.Retrieval.VectorWeight = x
    s.Retrieval.TopK = 20 // hidden normalization
    return s
}
```

Put-Put may still hold, but Get-Put fails whenever the original TopK was not 20. Writing the current vector weight back changes another part of the configuration. This is not a valid optic for a campaign claiming to mutate only vector weight.

Exercises 2.1

1. Prove the three lens laws for a lens focusing on the first component of a pair (A, B) .
2. Write a lens focusing on `ChunkSpec.Size`. What validation conditions depend on `Overlap`, and how do they affect lawful updates?

3. Give an example of a useful configuration transformation that should *not* be presented as a lens because it intentionally changes multiple dependent fields.
4. Explain how lens composition helps a product-level optimizer use a parameter exposed by a deeply nested plugin.

2.3 From lenses to optics

Motivation

A lens assumes a product-like structure: read one part and update it. But optimization spaces contain other shapes.

A model configuration may select one of several backend variants, requiring a **prism**-like focus on one branch of a sum. A traversal may update many homogeneous parameters. A partial optic may focus only when a capability exists. A bidirectional learning system may propagate information backward through a computation.

Rather than baking every access pattern into the core, we want a general family of bidirectional compositional interfaces.

Definition 2.2 — Optic, informal

An **optic** is a composable abstraction for bidirectional access through a structured system. Lenses, prisms, traversals, and related constructions can be understood as special cases of an optic construction over suitable monoidal categories [Riley, 2018].

The fully general coend definition is useful in category theory but unnecessary for the first software kernel. What matters architecturally is that:

1. optics compose;
2. lawfulness can be stated independently of one concrete representation;
3. different domain plugins can expose different kinds of focus while sharing a common composition theory.

For the initial RAG optimizer, ordinary lawful lenses cover most “replace one component of a release specification” interventions. The architecture should nevertheless avoid naming the global extension point `Setter` if future interventions will need more structure.

Side topic — The optic coend

A common abstract form for an optic from (S, S') to (A, A') in a monoidal category is built from pairs of morphisms

$$S \rightarrow M \otimes A, \quad M \otimes A' \rightarrow S'$$

modulo a suitable equivalence over the residual object M . Intuitively, the forward direction exposes a focus A plus residual context M ; the backward direction uses that residual context to rebuild S' from an updated A' .

For a simple lens, the residual context is “the rest of the structure.” The abstract definition matters because it explains why many bidirectional interfaces compose by the same pattern.

Worked example 2.3 — A backend prism

Suppose:

```
type VectorBackend interface{ isVectorBackend() }

type ExactSQLite struct { /* ... */ }
```

```
type HNSW struct {
  EfSearch int
  M        int
}
```

An optimizer wishing to tune `EfSearch` can only do so if the active backend is HNSW. A partial optic or prism-like interface is more honest than a lens that invents an HNSW value when the backend is exact.

```
type Focus[A any] struct {
  Value A
  OK    bool
}
```

The domain plugin decides what changing backend *kind* means and which larger dependency closure it invalidates.

Counterexample 2.2 — One universal reflection-based setter

A reflection API such as

```
SetPath(spec, "retrieval.backends[1].ef_search", 80)
```

can be convenient at a UI boundary. It should not be the semantic intervention primitive. It cannot naturally express branch legality, lawfulness, parameter-specific validation, or the causal declaration attached to the change.

2.4 Interventions are richer than values

Motivation

Even a lawful lens plus a replacement value is not enough to characterize an experiment. We also need to know what the change *claims* to do.

Changing `vector_weight` has no build-time invalidation. Changing `chunk_size` changes every chunk-dependent artifact. Changing a reranker model may alter network disclosure and provider cost. An optimizer needs a typed intervention record that couples the local update with its semantic declaration.

Definition 2.3 — Intervention

An **intervention** is a tuple

$$I = (L, a', \kappa, \Delta, \Phi),$$

where:

- L is an optic or lawful focus into the baseline parameter object;
- a' is the proposed replacement or transformation;
- κ is the **semantic class** of the intervention;
- Δ is a declared or computed **dependency/invalidation closure**;
- Φ is a collection of claims or proof obligations that must hold.

Typical semantic classes include:

- **operational**: intended to preserve ideal outputs while changing execution characteristics;
- **approximation**: replaces an exact computation with an approximate one;
- **relevance**: changes ranking or evidence selection;
- **knowledge**: changes the material indexed or generated representations;
- **policy/security**: changes authorization or disclosure behavior;
- **interaction**: changes agent or multi-turn dynamics;

- **presentation:** changes user-visible projection.

These classes are not mutually exclusive.

Worked example 2.4 — ANN efSearch

An HNSW efSearch change might be recorded as:

```
Intervention{
  Optic: "release.vector.hnsw.ef_search/v1",
  Value: 100,
  Classes: []Class{Approximation, Operational},
  Targets: []Node{"vector.query"},
  Claims: []Claim{
    RecallAtLeast(0.98),
    NoUnauthorizedResult(),
    P95LatencyImproves(),
  },
}
```

No embedding or index rebuild is required if efSearch is query-time only. A change to M or efConstruction would target build-time nodes and force a new vector index.

Worked example 2.5 — Chunking

Changing chunk size is a different intervention:

```
Intervention{
  Optic: "release.chunking.size/v1",
  Value: 800,
  Classes: []Class{Knowledge, Relevance},
}
```

Its invalidation closure includes chunks, generated representations derived from changed chunks, embeddings, lexical/vector indexes, and downstream evaluations that refer to chunk identities. The original source snapshot and normalization may remain reusable.

This difference is too important to leave as knowledge encoded only in build scripts.

2.5 Plugins as algebraic signatures

Motivation

We now arrive at the central extensibility question. The core must be small, but RAG needs chunkers, indexes, retrievers, rerankers, evaluators, and intervention spaces. A compiler optimizer needs passes, flags, benchmarks, and code-size metrics. A scheduling optimizer needs policies, simulations, and service-level constraints.

The conventional answer is “plugins.” But the word *plugin* often means little more than “code loaded elsewhere.” A callback interface can make the core smaller syntactically while making semantics weaker.

We need a stronger interpretation:

A plugin extends the *signature* of the optimization language and provides implementations and laws for the generators it introduces.

Definition 2.4 — Algebraic signature

An **algebraic signature** consists of sorts/types together with operation symbols and their arities. For example, the signature

$$\text{chunk} : \text{Spec} \otimes \text{Corpus} \rightarrow \text{Chunks}$$

and

$$\text{index} : \text{Chunks} \rightarrow \text{Index}$$

declares operations without saying how they execute.

A **model** or **algebra** of the signature assigns concrete meanings to the sorts and operations while respecting the equations/laws of the theory.

In software terms, a plugin can contribute:

1. schemas or typed ports;
2. primitive operation descriptors;
3. codecs/canonicalization rules;
4. parameter spaces and optics;
5. evaluators and metric definitions;
6. capabilities and effects;
7. law suites;
8. optional interpreters or adapters.

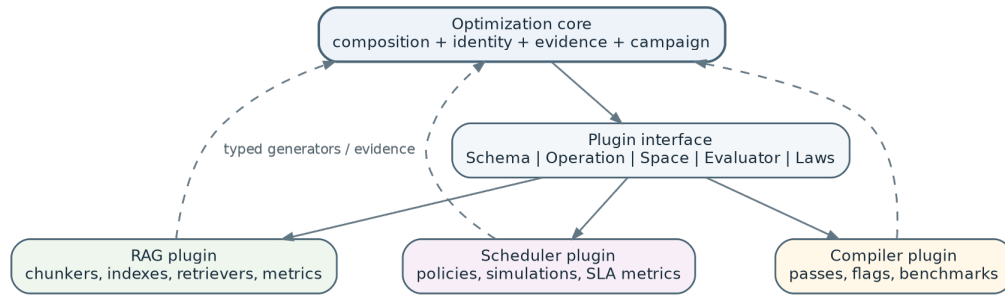


Figure 7: The optimization kernel remains domain-neutral while plugins graft typed signatures and laws onto it.

Worked API 2.1 — Plugin contract

```

type Plugin interface {
  Manifest() Manifest
  Install(*Builder) error
  Laws() []Law
}

type Manifest struct {
  ID          string
  Version     string
  Description string
}

type Builder interface {
  RegisterSchema(Schema) error
  RegisterOperation(Operation) error
  RegisterOptic(OpticDescriptor) error
  RegisterEvaluator(EvaluatorDescriptor) error
}
  
```

A deliberately narrow Builder is preferable to passing the whole engine into Install. The plugin is allowed to extend the signature, not to rewrite campaign state or bypass registries.

Definition 2.5 — Operation descriptor

An **operation descriptor** is the static semantic declaration associated with one primitive generator. A useful descriptor includes:

```
type OperationDescriptor struct {
    ID            OperationID
    Version       string
    Plugin        string
    Inputs        Port
    Outputs       Port
    Effects       []Effect
    Deterministic bool
    Cacheable     bool
    Dependencies  []DependencyID
    Cost          CostHint
}
```

The runtime implementation is registered separately or as part of the operation value, but plan analysis needs only the descriptor.

Why plugins should declare laws

A plugin operation may claim determinism, cacheability, or a lawful optic. Those claims should have executable tests.

Examples:

- canonical encoding round-trips;
- operation output validates against its declared schema;
- a deterministic operation produces identical material outputs for the same semantic inputs in a controlled test;
- an optic satisfies its laws;
- an incremental backend agrees with a full-build oracle;
- a ranking comparator is total over all valid finite scores;
- authorization filtering happens before remote disclosure.

A plugin without laws is not necessarily unsafe, but its semantic claims remain unverified assumptions.

Design callout: plugins are not trusted merely because they are “inside the process.” A plugin is a semantic boundary. The core should validate its descriptors, schemas, effects, and declared laws just as it validates external inputs.

Counterexample 2.3 — The god-object plugin

```
type Plugin interface {
    Run(engine *Engine, arbitrary any) error
}
```

This interface is maximally flexible and minimally compositional. A plugin can mutate campaign state, create undeclared network calls, invent metrics after observing results, or write arbitrary artifacts. Static analysis cannot see through it.

The algebraic-signature approach gives plugins more *domain authority* and less *orchestration authority*.

Exercises 2.2

1. Design a plugin signature for a compiler optimization domain. Include at least three schemas and four primitive operations.
2. For each operation, list effects and determinism claims.
3. Write three laws the plugin should provide.
4. Explain why `Install(*Engine)` is a weaker abstraction than `Install(*SignatureBuilder)`.

2.6 Typed envelopes and the boundary between static and dynamic typing

Motivation

Go generics are useful within one compiled plugin, but a dynamic plugin registry may need to store values whose concrete types are not known to the core. We therefore need a runtime envelope that preserves a schema identity and a stable material identity.

Definition 2.6 — Typed envelope

A **typed envelope** is a pair of a schema identifier and canonical bytes, optionally with a content digest:

$$E = (\text{schema}, \text{bytes}, \text{digest}).$$

A codec owned by a plugin converts between a concrete language type and the envelope.

```
type Envelope struct {
    Schema SchemaID
    Data   []byte
    Digest Digest
}
```

The core uses `Schema` to validate ports and `Digest` for identity and artifact references. Domain code uses a typed codec:

```
type Codec[T any] interface {
    Schema() SchemaID
    Encode(T) (Envelope, error)
    Decode(Envelope) (T, error)
}
```

This gives us **static typing inside plugins** and **schema-checked dynamic composition across plugin boundaries**.

Worked example 2.6 — Crossing a plugin boundary

A RAG plugin may implement:

```
type ChunkSet struct { /* ... */ }
var ChunkSetCodec Codec[ChunkSet]
```

The execution engine stores only an `Envelope` on a plan wire. When the index operation runs, its adapter decodes the envelope as `ChunkSet`. If a plan wires a `Corpus/v1` envelope to the index operation, validation rejects the plan before execution.

Counterexample 2.4 — JSON without schema identity

Canonical JSON alone does not tell us what a value means. Two schemas can both serialize to `{"id":"x"}`. A schema ID must be part of the semantic type and, normally, part of the domain-separated content identity.

2.7 Canonical identity and content-addressed semantics

Motivation

Optimization campaigns need reproducibility. If the same baseline specification is serialized with map keys in a different order, it should not become a new semantic candidate. If a prompt, model version, or chunking policy changes, the identity should change. We need a precise distinction between **semantic identity** and incidental execution identity.

Definition 2.7 — Canonical encoding

A **canonical encoding** is a deterministic mapping

$$\text{encode} : X \rightarrow \{0, 1\}^*$$

such that values considered equal by the schema are encoded identically.

A content identity can then be domain-separated:

$$\text{ID}_X(x) = H(\text{"schema:X/v1"} \parallel \text{encode}(x)).$$

The domain separator prevents identical bytes representing different semantic kinds from colliding at the application-identity level even if the underlying cryptographic hash is the same.

Definition 2.8 — Semantic versus execution identity

Semantic identity includes inputs that may change the declared meaning of a computation: model version, prompt, corpus snapshot, ranking policy, authorization policy, metric definition.

Execution identity includes operational coordinates that should not change semantic meaning: attempt number, worker ID, machine, scheduling order, retry backoff.

This distinction is crucial for caching. Worker count should not invalidate an embedding cache. Embedding-model identity must.

Worked example 2.7 — Plan ID

A canonical plan ID might hash:

- normalized plan syntax;
- primitive operation IDs and semantic versions;
- schema IDs on ports;
- explicit static configuration bound into the plan.

It should not hash:

- process ID;
- current timestamp;
- thread count unless concurrency changes semantics;
- output-directory path.

Counterexample 2.5 — Path as identity

If an index release is identified by `/srv/index/current`, overwriting that directory changes behavior without changing identity. A location can point to an artifact, but it should not *be* the artifact's semantic identity.

2.8 Multiple semantics by folding the free plan

Motivation

The largest payoff from the free-plan architecture is that the core can answer domain-independent questions without teaching the executor special cases for every plugin.

Each interpreter is an algebra over the plan constructors.

Definition 2.9 — Plan algebra

For a result type R , a **plan algebra** provides meanings for each constructor:

$$\begin{aligned} \llbracket \text{id} \rrbracket_R &\in R, \\ \llbracket \text{primitive}(o) \rrbracket_R &\in R, \\ \llbracket \text{seq}(p_1, \dots, p_n) \rrbracket_R &= \text{seq}_R(\llbracket p_1 \rrbracket_R, \dots), \\ \llbracket \text{tensor}(p_1, \dots, p_n) \rrbracket_R &= \text{tensor}_R(\llbracket p_1 \rrbracket_R, \dots). \end{aligned}$$

Structural recursion gives a unique fold of a free syntax once the algebra is fixed.

Worked example 2.8 — Effect analysis

Let the carrier be a finite set of effects:

$$R = \mathcal{P}(\{\text{CPU}, \text{Network}, \text{Random}, \text{Read}, \text{Write}\}).$$

Map each primitive to its declared effect set. Interpret both sequence and tensor by union. The fold yields every possible effect in the plan.

This simple interpreter can answer “does this plan contain a remote call?” before execution.

Worked example 2.9 — Trust-boundary analysis

A more interesting carrier tracks a small abstract state:

```
type DisclosureState struct {
  Authorized bool
  RemoteText bool
  Violations []Violation
}
```

The authorize primitive sets Authorized; a remote text-bearing operation checks it. Sequence propagates state in order. Tensor analyzes branches. This interpreter can reject a plan structurally if source text may reach a remote reranker before authorization.

Notice how this differs from the set-of-effects analysis. “Network exists” is not itself a violation. The order between authorization and disclosure matters, so the interpreter is stateful over the syntax.

Worked example 2.10 — Provenance interpretation

A provenance interpreter can assign each primitive a node in a lineage graph and connect outputs to inputs according to plan wiring. Execution later fills material artifact IDs. The same free syntax thus serves as a scaffold for both prospective provenance (“what will depend on what?”) and retrospective provenance (“which exact artifact was produced?”).

Worked example 2.11 — Documentation as an interpreter

A graph renderer is also an interpreter. Primitive descriptors supply labels, plugin IDs, and port schemas. Sequence and tensor determine layout. This matters because architecture diagrams no longer need to be hand-maintained separately from executable plans.

Counterexample 2.6 — Every interpreter re-parses arbitrary code

If the “plan” is just ordinary Go functions, each analysis must recover structure through instrumentation, reflection, tracing, or duplicate metadata. The free syntax pays a small upfront cost to make many downstream analyses exact and reusable.

2.9 Effect systems: annotations versus semantics

Motivation

Real optimization operations use network calls, random generators, stores, clocks, and humans. We need to represent effects, but there are two different goals:

1. **classification**: know that an operation can perform an effect;
2. **semantic control**: model how effects compose and are interpreted.

A first implementation can accomplish much with descriptors. A more formal system may use monads, algebraic effects, graded effects, or capability-indexed morphisms.

Definition 2.10 — Effect annotation

An **effect annotation** is static metadata attached to an operation, for example:

```
type Effect string
```

```
const (
    CPU          Effect = "cpu"
    Network      Effect = "network"
    Random       Effect = "random"
    ArtifactRead Effect = "artifact-read"
    ArtifactWrite Effect = "artifact-write"
)
```

Annotations support preflight analysis but do not themselves define behavior.

Side topic — Why not make the whole architecture “a monad”?

Monads are a powerful way to sequence effectful computations. Moggi’s computational semantics explains many programming effects through monadic structure. But an optimization architecture needs more than sequential effects. It also needs explicit parallel composition, local parameter interventions, stochastic couplings, plan inspection, and multiple interpreters.

A monad may therefore appear **inside one interpreter**—for example, the execution interpreter might target IO, Result, or a task effect—but it need not be the top-level architecture. The free monoidal plan is a more neutral syntax.

Side topic — Algebraic effects

Algebraic effects separate operations such as ReadArtifact, CallModel, or EmitObservation from handlers that interpret them. This fits the same philosophy as free plans: preserve syntax until an interpreter is chosen. A future kernel can use an algebraic-effect representation inside primitive execution without changing the plugin-level plan theory.

2.10 Dependency semantics and invalidation

Motivation

Optimization differs from ordinary orchestration because changing a parameter can invalidate some existing artifacts while leaving others reusable. We need a semantics for causal dependency.

Let N be a set of semantic nodes such as:

```
source.snapshot
corpus.normalized
chunk.records
representations
embeddings
index.lexical
index.vector
query.fusion
reranker
answer.policy
retrieval.eval
session.eval
```

A plugin operation declares which nodes it reads and produces. An intervention targets one or more nodes. The invalidation closure is the set of downstream nodes reachable in the dependency graph.

Definition 2.11 — Dependency graph

A **dependency graph** is a directed graph $G = (V, E)$ whose vertices are semantic artifacts or policies and where $u \rightarrow v$ means that a change in u may change the semantic value of v .

For a changed set $S \subseteq V$, its **forward closure** is

$$\text{cl}^+(S) = \{v \mid \exists s \in S \text{ with a path } s \rightsquigarrow v\}.$$

A candidate must recompute or re-evaluate every material node in the closure unless a stronger plugin-specific equivalence proves reuse valid.

Worked example 2.12 — Chunker invalidation

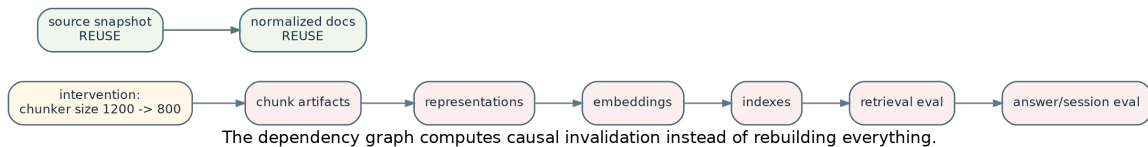


Figure 8: Changing chunk size invalidates chunk-derived artifacts while leaving the source snapshot and normalized documents reusable.

If only chunk size changes, the source snapshot and normalized documents can remain identical. Chunk records change; downstream generated representations and embeddings may change; indexes change; retrieval and answer evaluations must be rerun.

If only vector fusion weight changes, the closure begins at query fusion. All build artifacts remain reusable.

Why a dependency graph is not enough by itself

Dependencies can be conditional. An answer-only metric may not depend on the internal rank trace if it consumes only the final answer, while a diagnostic metric does. A representation change may not

affect lexical search if that representation kind is excluded from lexical indexing. Plugins therefore need versioned dependency declarations associated with the actual release plan, not one universal handwritten graph.

Counterexample 2.7 — “Rebuild all” as correctness strategy

Rebuilding everything is safe only if every build is deterministic and the environment is perfectly controlled. In stochastic or provider-backed stages, unnecessary rebuilds can introduce new sampled representations or embeddings and weaken paired comparisons. They also waste time and cost. Causal reuse is an experimental-design feature, not merely an optimization.

2.11 A concrete plugin: toy RAG as a grafted domain

The following miniature signature is sufficient to illustrate the architecture without making the core aware of RAG:

```
chunk    : Spec × Corpus -> ChunkSet
index    : ChunkSet -> Index
retrieve: Index × Spec × Case -> RetrievalResult
measure  : RetrievalResult -> TrialOutput
```

The plugin registers four schemas and four operations. Its concrete types might be:

```
type Spec struct {
  ChunkWords    int
  OverlapWords  int
  LexicalWeight float64
  VectorWeight  float64
  TopK          int
}

type CaseInput struct {
  Query           string
  RelevantDocuments []string
}

type TrialOutput struct {
  Recall    float64
  MRR       float64
  HitRate   float64
  ScoredChunks float64
  ContextWords float64
}
```

The core sees only schema IDs, operation descriptors, plans, trial envelopes, and metrics. The RAG plugin owns tokenization, chunking, vectorization, retrieval, and metric semantics.

Worked example 2.13 — Registering a primitive

```
builder.RegisterOperation(Binary[Spec, Corpus, ChunkSet]{
  Desc: OperationDescriptor{
    ID: "ragtoy.chunk/v1",
    Inputs: Port{SchemaSpec, SchemaCorpus},
    Outputs: Port{SchemaChunks},
    Effects: []Effect{CPU},
    Deterministic: true,
    Cacheable: true,
    Dependencies: []string{"corpus.normalized", "index.chunk"},
  },
},
```

```
Run: chunkCorpus,
})
```

The operation’s implementation can be replaced or moved to another package without changing the plan vocabulary as long as its semantic version and laws remain valid.

Worked example 2.14 — The build and query plans

Build:

$$(\text{Spec} \otimes \text{Corpus}) \xrightarrow{\text{chunk}} \text{Chunks} \xrightarrow{\text{index}} \text{Index}.$$

Query evaluation:

$$\text{Index} \otimes \text{Spec} \otimes \text{Case} \xrightarrow{\text{retrieve}} \text{Retrieval} \xrightarrow{\text{measure}} \text{Trial}.$$

The campaign runner may build once per build-affecting candidate and reuse the resulting index for many cases. This scheduling policy is a higher-level interpretation of dependencies, not something encoded inside retrieve.

2.12 Plugin law testing

A plugin’s `Laws()` method should provide fast, deterministic checks for its semantic claims.

Example 2.15 — Lens laws

```
func CheckLensLaws[S, A any](
  l Lens[S, A],
  states []S,
  values []A,
) error
```

Production code should prefer property-based generators over small fixed lists where possible.

Example 2.16 — Operation determinism

For a deterministic operation `f`, test:

$$\text{material}(f(x)) = \text{material}(f(x))$$

across separate executions in a controlled environment. This is not a proof that a network-backed operation is deterministic merely because two calls happened to match. The plugin should only claim determinism when the semantics justify it.

Example 2.17 — Plan typing

Generate random well-typed plans from the plugin signature and check that every interpreter accepts them. Mutate one port schema and check that validation rejects the plan before execution.

Exercise 2.3 — Design a law suite

For a vector-index backend plugin, propose laws or conformance tests for:

1. insert/search;
2. delete/tombstone;
3. filter soundness;

4. total score order;
5. reopen from artifact;
6. approximate recall relative to an exact oracle;
7. stable semantic identity.

Classify which are algebraic laws, which are deterministic tests, and which are statistical claims.

2.13 Counterexample clinic: three extensibility traps

Trap A — Callbacks without descriptors

```
type Evaluator func(context.Context, any) (float64, error)
```

The core cannot know inputs, effects, stochasticity, metric direction, failure semantics, or dependency scope. The function may still be a useful internal adapter, but it is too weak as the public semantic contract.

Trap B — Interfaces that mirror one implementation

```
type VectorPlugin interface {
    OpenSQLite(path string) error
    SQLQuery() string
}
```

This interface is “generic” only in name. It exposes the current backend rather than the domain capability. A better contract describes Build, Open, Search, Filter, Delete, and the backend’s declared capabilities.

Trap C — A plugin owns the campaign loop

If each plugin implements its own Optimize() method, cross-domain composition disappears. The compiler plugin, RAG plugin, and scheduling plugin each invent separate evidence, retry, pairing, and decision semantics. The shared core becomes a registry of unrelated mini-frameworks.

The plugin should own *domain meaning*; the campaign kernel should own *experimental custody*.

2.14 Chapter synthesis: the small semantic kernel

By the end of Chapter 2, the architecture has a surprisingly compact center:

```
core/
  schema + canonical envelope
  content identity
  primitive descriptors
  free typed plan
  plan fold/interpreter interface
  effect/dependency/cost carriers
  evidence/result envelope

extension interfaces/
  plugin signature builder
  typed codecs
  optics/intervention spaces
  evaluator descriptors
  laws and capability tests
```

Nothing here says Chunk, HNSW, Prompt, CompilerFlag, or SLA. Domain plugins graft those concepts onto the signature.

The strength of the architecture comes from restricting extension points. A plugin does not get an arbitrary callback into the optimizer. It gets named places to contribute types, generators, focuses, evaluators, capabilities, and laws. This resembles adding operations to a theory more than injecting code into a framework.

Chapter 2 review problems

1. A plugin exposes a lens that updates `EmbeddingModel` but silently rewrites `EmbeddingDimension` to the model's default. Decide whether this should be one lens, two coupled parameters, or a higher-level transformation. Justify your design using lens laws.
2. Define a plan interpreter that computes the *maximum trust level* of data that can reach each remote operation.
3. Design a canonical schema-evolution rule for `RetrievalSpec/v1 -> v2`. When should two versions intentionally have different content IDs even if their JSON happens to match?
4. Show how a product-level release spec can compose optics exported by independent chunking and reranking plugins.
5. Compare “plugins as algebraic signatures” with dependency injection. What problems do they solve in common, and what additional semantic commitments does the signature view add?
6. Write pseudocode for an invalidation planner that takes an intervention target set and a release-specific dependency DAG and returns reuse, rebuild, and reevaluate sets.

Chapter 3. Probability, Experiments, Decisions, and Feedback

3.1 Why deterministic semantics are not enough

Motivation

The process language of Chapters 1 and 2 is useful even when every stage is deterministic. Real optimization fields are rarely so simple. The same prompt and model configuration can produce different answers. A remote reranker can vary under provider updates or floating-point nondeterminism. Human judgments differ across assessors. Production latency changes with load. Even a deterministic retriever is evaluated on a sampled workload whose composition is uncertain relative to future traffic.

The common engineering response is to say that “evaluation is noisy.” That phrase is too weak. We need to represent:

- what is random;
- what is conditioned on the candidate and case;
- which randomness is shared between baseline and candidate;
- what evidence is retained from each sample;
- which statistical comparison is valid.

Probability should therefore enter the semantics, not only the reporting layer.

3.2 Markov kernels

Definition 3.1 — Probability distribution

For a finite set Y , a **probability distribution** is a function

$$p : Y \rightarrow [0, 1]$$

such that

$$\sum_{y \in Y} p(y) = 1.$$

We write $\mathcal{D}(Y)$ for the set of distributions on Y .

Definition 3.2 — Markov kernel

A **Markov kernel** from X to Y assigns a distribution on Y to every input $x \in X$:

$$K : X \rightarrow \mathcal{D}(Y).$$

Equivalently, we write

$$K(y \mid x)$$

for the probability of output y conditioned on input x .

In a finite implementation:

```
type Dist[T comparable] map[T]float64
```

```
type Kernel[A, B comparable] func(A) Dist[B]
```

A language-model evaluator, a latency model, or a sampled human assessor can all be treated as kernels once their conditioning inputs are stated explicitly.

Worked example 3.1 — A stochastic answer evaluator

Let C be candidate releases, X evaluation cases, and O trial outcomes. Then evaluation can be modeled as

$$K : C \times X \rightarrow \mathcal{D}(O).$$

An outcome should be richer than a score:

```
type Outcome struct {
  Status      Status
  Metrics      MetricVector
  Trace        TraceRef
  Artifacts    []ArtifactRef
  Failure      *Failure
}
```

This design preserves failure and provenance as part of the sampled evidence.

Definition 3.3 — Kleisli composition for probability

Given kernels

$$K : X \rightarrow \mathcal{D}(Y)$$

and

$$L : Y \rightarrow \mathcal{D}(Z),$$

their composite is

$$(L \odot K)(z \mid x) = \sum_{y \in Y} K(y \mid x) L(z \mid y).$$

This is the law of total probability expressed as composition. It is composition in the **Kleisli category** of the distribution monad.

Fundamentals: monads and Kleisli arrows. For this chapter it is enough to think of the distribution construction $\mathcal{D}$ as turning a set of outcomes into a set of probability distributions. A Kleisli arrow $X \rightarrow Y$ for $\mathcal{D}$ is an ordinary function $X \rightarrow \mathcal{D}(Y)$. Kleisli composition performs the summation/integration needed to feed a random output of the first arrow into the second. The general definition of a monad packages the unit and associative “flattening” operations that make this composition lawful.

Worked example 3.2 — Retrieval followed by stochastic generation

Suppose retrieval is deterministic:

$$r : Q \rightarrow E,$$

while generation is stochastic:

$$g : E \rightarrow \mathcal{D}(A).$$

Then the whole answer system is a kernel

$$Q \rightarrow \mathcal{D}(A)$$

given by

$$q \mapsto g(r(q)).$$

If retrieval itself uses stochastic query expansion, reranking, or connected search, the same composition law handles it.

Counterexample 3.1 — Averaging away the failure state

Suppose five answer trials yield quality scores

0.9, 0.8, FAILED, 0.85, FAILED

A reporting function that drops failures and reports mean 0.85 changes the probability space. The system did not produce a quality score on 40% of trials. A correct outcome kernel includes failure as an explicit branch or defines a total metric with a justified penalty.

Exercises 3.1

1. Let K flip a fair coin and L output win with probability 0.8 after heads and 0.3 after tails. Compute $(L \odot K)(\text{win})$.
2. Model a retrieval system with deterministic retrieval and a generator that abstains with probability depending on evidence count.
3. Explain why treating “exception” as absence of a sample rather than an outcome can bias optimization.

3.3 Markov categories: probability plus compositional wiring

Motivation

Kleisli categories explain sequential composition of kernels, but our plan language also needs parallel composition, copying, and discarding. Categorical probability packages these structural operations into **Markov categories**. Tobias Fritz’s framework gives a synthetic language for probability that is independent of one particular measure-theoretic representation [Fritz, 2020].

Definition 3.4 — Markov category, operational version

A **Markov category** is, roughly, a symmetric monoidal category of stochastic processes equipped with coherent copying and discarding operations on objects, with discarding compatible with normalization.

Each object X has a copy morphism

$$\text{copy}_X : X \rightarrow X \otimes X$$

and discard morphism

$$\text{discard}_X : X \rightarrow I.$$

Deterministic morphisms interact with copying in the familiar way. General stochastic morphisms do not: sampling once and copying the result is not the same as independently running the kernel twice.

The full axioms are given in the categorical-probability literature [Fritz, 2020]. For software architecture, the key distinction is between **copying information already obtained** and **duplicating a stochastic computation**.

Worked example 3.3 — One sample versus two samples

Let $K : I \rightarrow \{H, T\}$ be a fair coin.

Sample once, then copy:

$$I \xrightarrow{K} X \xrightarrow{\text{copy}} X \otimes X.$$

The joint distribution is

$$P(H, H) = 1/2, \quad P(T, T) = 1/2,$$

with zero probability on (H,T) and (T,H).

Run twice independently:

$$I \cong I \otimes I \xrightarrow{K \otimes K} X \otimes X.$$

Now every pair has probability 1/4.

This difference reappears in experiments. If baseline and candidate are evaluated with independent random seeds, we obtain one coupling. If they share the same stochastic scenario, we obtain another.

Fundamentals: copying a random variable is not resampling. A random variable is a realized value; a kernel is a stochastic mechanism. This distinction is easy to blur in code when both are represented by functions using an ambient random generator.

Worked example 3.4 — Randomness as an explicit input

A practical runtime can make stochasticity reproducible by exposing a seed:

```
type TrialRequest struct {
    Candidate CandidateID
    Case       CaseID
    Repeat     int
    Seed       Seed
}
```

The evaluator may still call external systems that do not guarantee deterministic replay, but seed identity is part of the experimental coordinate. Randomness that *can* be controlled is no longer hidden in global state.

3.4 Couplings: the mathematics of paired experiments

Motivation

Suppose we compare baseline B and candidate C . Evaluating them on the same query set is good, but for stochastic systems we also need to decide how their randomness is related.

If each arm receives independent random conditions, the difference between outcomes includes both treatment effect and unrelated stochastic variation. If the same random condition can be meaningfully shared, a paired design often has lower variance.

The mathematical object describing the relationship between two marginal distributions is a **coupling**.

Definition 3.5 — Coupling

Let $\mu \in \mathcal{D}(X)$ and $\nu \in \mathcal{D}(Y)$. A **coupling** of μ and ν is a joint distribution

$$\gamma \in \mathcal{D}(X \times Y)$$

whose marginals are μ and ν .

Different couplings can have the same marginals but different dependence structures.

Worked example 3.5 — Shared seeds as a coupling

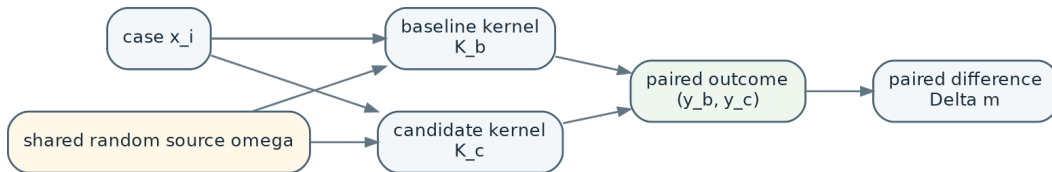
For case x_i and repeat r , derive one deterministic seed

$$\omega_{i,r} = H(\text{campaignID}, i, r).$$

Evaluate both arms using sub-seeds deterministically derived from the same root:

$$Y_B \sim K_B(x_i, \omega_{i,r}), \quad Y_C \sim K_C(x_i, \omega_{i,r}).$$

This does not guarantee identical provider randomness; many providers ignore user seeds or have hidden state. But it defines the intended coupling for every controllable source of randomness.



A coupling is part of experiment semantics, not an implementation detail.

Figure 9: Baseline and candidate evaluations can be coupled through a shared stochastic coordinate.

Definition 3.6 — Paired difference

For metric m , the paired difference on one coordinate is

$$\Delta_{i,r} = m(Y_C^{i,r}) - m(Y_B^{i,r}).$$

The empirical mean paired effect is

$$\bar{\Delta} = \frac{1}{N} \sum_{i,r} \Delta_{i,r}.$$

For a minimized metric such as latency or error, the sign convention may be reversed or metric direction recorded separately.

Worked example 3.6 — Why pairing can reduce variance

Suppose cases have very different difficulty. Baseline and candidate quality can be modeled as

$$B_i = \theta_B + d_i + \epsilon_{B,i},$$

$$C_i = \theta_C + d_i + \epsilon_{C,i},$$

where d_i is case difficulty shared by both arms. The paired difference cancels d_i :

$$C_i - B_i = (\theta_C - \theta_B) + (\epsilon_{C,i} - \epsilon_{B,i}).$$

An unpaired comparison must estimate through the variability in d_i .

Counterexample 3.2 — Invalid sharing of randomness

Shared randomness is not automatically better. If the candidate changes the sampling space itself, forcing identical low-level random choices can create an artificial coupling.

Example: baseline chunks a document into 20 pieces and candidate chunks it into 35. “Use random chunk index 7 in both arms” does not represent a shared semantic event. The correct shared coordinate may be the same query/case and provider seed, while chunk sampling is arm-specific.

The campaign design must share **semantically corresponding randomness**, not merely equal integers.

Exercises 3.2

1. Construct two different couplings of two fair coins: independent and perfectly correlated.
2. Which coupling would you use to compare two prompt variants on the same set of model seeds? What assumptions does this make?
3. Give a RAG intervention for which some sources of randomness can be paired and others cannot.
4. Explain why “same case ID” is necessary but not sufficient for a paired stochastic experiment.

3.5 Deterministic seed splitting

Motivation

A complex trial may need separate random streams for query rewriting, answer generation, judge sampling, and workload perturbation. Using one mutable pseudorandom generator makes execution order affect results. If we parallelize two stages, the random numbers consumed by each may change.

A better design derives independent named sub-seeds from an immutable root.

Definition 3.7 — Splittable seed discipline

Let H be a cryptographic hash or pseudorandom derivation. A root seed s can derive a sub-seed by label:

$$\text{split}(s, \ell) = H(s \parallel 0 \parallel \ell).$$

A trial can then use

```

root
  /rewrite
  /generation
  /judge/grounding
  /judge/style

```

without depending on call order.

Worked API 3.1

```
type Seed [32]byte
```

```
func (s Seed) Split(label string) Seed
func (s Seed) Rand() *rand.Rand
```

A campaign derives root seeds from exact coordinates:

```
seed := SeedFromString(
    campaignID + "/" + caseID + "/" + strconv.Itoa(repeat),
)
```

Changing worker count or arm execution order does not change the controlled random streams.

Counterexample 3.3 — Global RNG

```
rand.Seed(42)
```

followed by concurrent evaluation makes results depend on scheduling. Even with a lock, adding one random call to a logging path can shift every subsequent trial. Global RNG state is execution identity contaminating experiment semantics.

3.6 Metrics are observations, not decisions

Motivation

Optimization systems frequently collapse evaluation into a number called score. This is attractive because many search algorithms expect a scalar objective. But products usually care about many incompatible dimensions:

- recall should increase;
- latency should decrease;
- unauthorized disclosure must remain zero;
- failure rate must not regress;
- answer groundedness must remain above a floor;
- cost is often a tie-break, not a license to sacrifice safety.

A metric vector is evidence. A decision policy says how that evidence is used.

Definition 3.8 — Metric definition

A metric definition includes at least:

$$M = (\text{id}, \text{direction}, \text{domain}, \text{missingPolicy}, \text{aggregation}).$$

The **direction** is maximize, minimize, or sometimes constraint-only.

A trial emits a partial or total metric vector

$$m : \text{MetricID} \rightarrow \mathbb{R}.$$

Missingness must have explicit semantics. It must never be silently converted to zero or omitted from denominators unless the metric definition says so.

Worked example 3.7 — RAG trial metrics

retrieval.recall@10	maximize
retrieval.mrr	maximize
answer.groundedness	maximize
security.disclosures	constraint: == 0
runtime.p95_latency_ms	minimize
runtime.provider_calls	minimize
cost.usd	minimize

A candidate can dominate another on some metrics and lose on others.

Definition 3.9 — Preorder and partial order

A **preorder** is a relation $\preceq$ that is reflexive and transitive. A **partial order** is additionally antisymmetric: if $x \preceq y$ and $y \preceq x$, then $x = y$. Optimization preferences are often preorders rather than total orders because two candidates can be incomparable: one is faster while another is more accurate.

A **total order** adds comparability: for every x, y , either $x \preceq y$ or $y \preceq x$. A single scalar score induces a total preorder, which is convenient but frequently stronger than product policy justifies.

Definition 3.10 — Pareto dominance

For a set of metrics transformed so that larger is better, candidate vector x **Pareto-dominates** y when

$$\forall j, x_j \geq y_j$$

and

$$\exists j, x_j > y_j.$$

The **Pareto frontier** is the set of candidates not dominated by another candidate.

Pareto analysis avoids inventing exchange rates between incommensurable goals. It does not itself choose a final candidate.

Counterexample 3.4 — Universal weighted score

Suppose

$$S = 10 \cdot \text{quality} - 0.001 \cdot \text{latency} - 100 \cdot \text{disclosures}.$$

The weights imply that a sufficiently large quality gain can compensate for unauthorized disclosure. If the product requirement is “zero unauthorized disclosures,” the score encodes the wrong decision topology. Hard constraints must be represented as hard constraints.

3.7 Ordered gates and decision algebras

Motivation

Many promotion decisions are naturally staged:

1. Is the experiment complete and valid?
2. Did any safety or integrity invariant fail?
3. Did the target metric improve enough?
4. Did protected strata remain noninferior?
5. Is cost acceptable?

The order matters. We need a decision policy that is compositional but does not pretend every condition is a scalar objective.

Definition 3.11 — Gate

A **gate** is a predicate or statistical decision

$$g : E \rightarrow \{\text{pass, fail, insufficient}\}$$

on accumulated experiment evidence E .

A **gate sequence** evaluates gates in a declared order. A fail-closed policy stops eligibility at the first failure or insufficient hard gate.

Worked example 3.8 — RAG promotion sequence

```
G1 coverage:
  every baseline/candidate/case/repeat cell is present
G2 security:
  unauthorized_remote_disclosure == 0
G3 integrity:
  invalid_grounding_rate <= 0.001
G4 target:
  paired recall improvement lower bound > 0
G5 protected strata:
  no role/source/query-class regression beyond margin
G6 operational:
  p95 latency <= budget
G7 preference:
  choose lower cost among eligible Pareto candidates
```

This policy is understandable because every gate corresponds to a product claim.

Definition 3.12 — Lexicographic preorder

If metric groups have strict priority, we can define a lexicographic preorder. Compare candidates first on the highest-priority group; only ties or acceptable equivalence proceed to lower groups.

Hard gates are even stronger: an infeasible candidate is outside the eligible set rather than merely worse.

Worked API 3.2

```
type GateResult struct {
  Status  GateStatus // pass, fail, insufficient
  Evidence []ArtifactRef
  Reason  string
}

type Gate interface {
  ID() string
  Evaluate(Comparison) GateResult
}
```

```

type Sequence struct {
  PolicyID string
  Gates    []Gate
}

```

Counterexample 3.5 — Decision logic inside the evaluator

If an evaluator returns only `promote=true`, we cannot distinguish evidence from policy. Changing product tolerance requires rerunning evaluation or trusting hidden evaluator logic. A reusable architecture should retain native trial evidence and run a versioned decision algebra over it.

3.8 Statistical claims and uncertainty

Motivation

A paired mean difference is an estimate, not a fact about future workload. A promotion gate should normally include uncertainty or a deliberately deterministic workload claim.

The architecture should not hard-code one statistical method, but it should make the sampling unit and paired coordinates explicit enough that valid methods can be applied.

Definition 3.13 — Estimand

An **estimand** is the population quantity an experiment seeks to estimate. For example:

$$\theta = \mathbb{E}_{X \sim W, \omega \sim \Omega} [m(C, X, \omega) - m(B, X, \omega)].$$

Here W is the target workload distribution and Ω the stochastic environment under the chosen coupling.

The sample mean is meaningful only relative to this declared population.

Worked example 3.9 — Paired bootstrap

Given case-level paired differences $\Delta_1, \dots, \Delta_n$, resample cases with replacement and recompute the mean to approximate a confidence interval. If repeats within a case share important dependence, resample at the case cluster level rather than treating every repeat as independent.

Pseudocode:

```

for b in 1..B:
  sampledCases = sample_with_replacement(cases, n)
  deltas = all paired deltas belonging to sampledCases
  bootstrapMean[b] = mean(deltas)
interval = percentile(bootstrapMean, [2.5%, 97.5%])

```

Definition 3.14 — Noninferiority

For a protected metric where regression up to margin δ is acceptable, a noninferiority claim tests whether

$$\theta > -\delta.$$

This is often a better product statement than “no statistically significant difference.” Failure to detect a difference is not evidence of equivalence.

Side topic: statistical significance is not product significance. A tiny latency increase can be statistically clear at large sample sizes and operationally irrelevant. A large quality gain can be uncertain because the evaluation set is small. Gates should combine effect size, uncertainty, and product margins.

Counterexample 3.6 — Repeated peeking

If a canary is checked every minute and stopped as soon as a conventional 95% interval happens to exclude zero, the nominal coverage no longer describes the procedure. Sequential evaluation requires a method designed for repeated looks or a conservative stopping policy.

3.9 Comparison of statistical experiments

Motivation

Sometimes the question is not merely “which candidate scored higher?” but “does one evaluation setup contain at least as much decision-relevant information as another?” Categorical probability provides a deep connection here through the comparison of statistical experiments and the Blackwell order. Fritz, Gonda, Perrone, and Rischel formulate such comparisons in Markov categories [Fritz et al., 2020].

We do not need the full theorem to design the software, but it gives a useful perspective on evaluators.

Definition 3.15 — Garbling, informal

An experiment E_2 is a **garbling** of E_1 if the observations of E_2 can be obtained by applying an additional stochastic channel to the observations of E_1 .

Then E_1 is at least as informative as E_2 for every decision problem in the classical Blackwell sense.

Worked example 3.10 — Native artifact versus projected score

Suppose a RAG evaluator produces a native artifact containing:

ranked candidates
channel contributions
reranker trace
answer
citations
provider usage
failure class

and then projects it to a metric vector (recall, groundedness, latency).

The metric vector is a deterministic garbling of the native artifact: it intentionally loses information. Therefore the native artifact should remain the evidence source for later diagnosis or alternative decision policies. The projection is sufficient for some gates, not necessarily for all future questions.

This observation strongly supports a two-level evaluator interface:

1. retain a product-native artifact;
2. project a stable cross-campaign metric envelope.

Exercises 3.3

1. Give an example where a scalar score is a garbling of a richer trial artifact.
2. Can two different native artifacts project to the same metric vector? What does this imply for debugging?

3. Why might an optimizer be allowed to see projected metrics but not the complete hidden evaluation artifact?

3.10 Campaigns as state machines

Motivation

So far we can build candidates and evaluate them. Optimization becomes a process when evidence accumulates over time and influences the next action.

A real campaign must also survive interruption. It should know which trial cells are complete, which failures occurred, which candidate was tested, and which decision was made. This naturally suggests a state-machine or coalgebraic view.

Definition 3.16 — Transition system

A **state transition system** consists of states S , labels/actions L , and a relation or function

$$S \xrightarrow{\ell} S'.$$

A deterministic reducer can be written

$$\text{reduce} : S \times E \rightarrow S$$

where E is an event type.

Definition 3.17 — Endofunctor

An **endofunctor** is a functor from a category to itself, $F : \mathcal{C} \rightarrow \mathcal{C}$. It describes a uniform “shape constructor” on objects and morphisms. Examples in programming include list-like, option-like, or state-transition shapes.

Definition 3.18 — Coalgebra

Given an endofunctor F , an **F -coalgebra** is a map

$$c : S \rightarrow F(S).$$

Coalgebras are widely used to model state-based, potentially ongoing behavior. For an optimizer, $F(S)$ can encode a next action plus continuation state or termination.

A simplified campaign coalgebra might be

$$c : S \rightarrow \text{Done} + \text{Action} \times S.$$

The exact functor is less important than the design insight: the optimizer is not a pure function from initial config to final config. It is an observable evolving process.

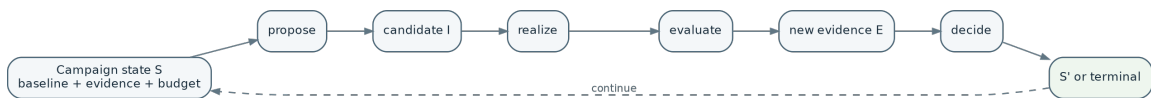


Figure 10: A campaign repeatedly proposes, realizes, evaluates, decides, and either terminates or returns a new state.

Worked example 3.11 — Campaign state

```

type CampaignState struct {
    CampaignID    ID
    Baseline      CandidateID
    Proposed      []CandidateID
    Trials        map[Coordinate]TrialResult
    Decisions     []DecisionRecord
    Budget        BudgetState
    Status        CampaignStatus
}

```

A coordinate might be:

```

type Coordinate struct {
    Candidate CandidateID
    Case      CaseID
    Repeat    int
    Arm       ArmID
}

```

An append-only event log records state transitions:

```

CampaignStarted
CandidateProposed
TrialStarted
TrialCompleted
TrialFailed
ComparisonComputed
GateEvaluated
CandidateAccepted
CampaignCompleted

```

The in-memory state is a pure reduction of the log.

Why an append-only log matters

Optimization evidence is scientific evidence. Overwriting `results.json` loses the history of missing cells, retries, and intermediate decisions. An append-only log plus immutable artifacts supports:

- resume;
- audit;
- deterministic reconstruction;
- idempotent re-execution;
- post-hoc diagnosis;
- testing the state reducer independently from I/O.

Definition 3.19 — Resumability law

Let $E = E_1 \cdot E_2$ be an event sequence split at any durable prefix. If

$$S_1 = \text{fold}(S_0, E_1),$$

then resuming from S_1 and applying E_2 should yield the same terminal state as uninterrupted reduction:

$$\text{fold}(S_1, E_2) = \text{fold}(S_0, E).$$

Execution may repeat some external work after a crash, but semantic trial coordinates and committed results must remain consistent.

Counterexample 3.7 — “Retry by deleting the run directory”

A campaign whose only recovery procedure is “delete partial output and start over” weakens experimental control. Provider calls may resample, expensive builds repeat, and partial failures disappear. A resumable campaign makes incomplete evidence explicit.

3.11 Campaigns as cybernetic systems

Motivation

The word *optimization* suggests a controller acting on a system through feedback. Categorical cybernetics studies compositional processes that interact with environments and controllers, including open learners [Capucci et al., 2021]. This perspective is useful when the proposer itself is adaptive or learned.

We can separate three roles:

- the **plant/system** being optimized;
- the **environment** supplying workloads and stochastic conditions;
- the **controller** proposing interventions based on evidence.

Definition 3.20 — Feedback optimizer, schematic

Let

$$\text{System} : P \otimes X \rightarrow \mathcal{D}(Y)$$

be the parameterized system, and let an evaluator convert outcomes into evidence E . A controller updates campaign state or parameters:

$$\text{Controller} : S \otimes E \rightarrow \mathcal{D}(S \otimes P').$$

The closed loop repeatedly composes these processes.

The categorical viewpoint is especially valuable when the controller itself is a plugin: a grid proposer, Bayesian search process, human reviewer, or LLM proposer can all consume the same evidence interface.

Worked example 3.12 — LLM self-optimization without semantic privilege

Suppose an LLM proposes modifications to a retrieval prompt. A safe architecture does **not** give the model arbitrary filesystem mutation. Instead:

1. the LLM outputs a proposed intervention in a constrained schema;
2. the plugin validates it against a lawful optic or transformation;
3. the dependency planner computes invalidation;
4. the campaign engine realizes artifacts;
5. fixed evaluators produce native evidence;
6. a fixed decision policy accepts/rejects;
7. the LLM sees only the permitted projection of evidence before proposing again.

The LLM is a controller inside the field, not the authority that defines the field.

Counterexample 3.8 — Self-judging self-optimization

A model proposes a prompt, runs itself on a few examples, judges its own outputs with an unversioned rubric, discards failures, edits the prompt file, and repeats. This is a feedback loop, but it has almost no experiment semantics. It may be useful for brainstorming; it is not sufficient evidence for controlled promotion.

3.12 Open games and decision boundaries

Motivation

Optimization systems sometimes involve multiple decision-makers: automated proposer, evaluator, product owner, safety gate, and deployment authority. Compositional game theory and open-game formalisms motivate treating decision components as open systems that interact through interfaces rather than collapsing them into one global objective.

We will not develop open-game theory fully, but one lesson is immediately practical:

A component can expose what information it receives, what choice it makes, and what feedback it observes without knowing the entire global utility function.

This aligns with plugin boundaries. A RAG evaluator emits evidence. A security gate emits eligibility. A cost policy emits preference among eligible candidates. A human review stage may observe a selected evidence view. Composition builds the final decision process.

Worked example 3.13 — Separate eligibility from choice

Let

$$\text{eligible} : C \times E \rightarrow \{0, 1\}$$

and let

$$\text{prefer} : \mathcal{P}(C) \times E \rightarrow C$$

choose among eligible candidates. This is structurally different from

$$\text{score} : C \times E \rightarrow \mathbb{R}.$$

The separation allows security to remain a hard feasibility condition while cost and quality participate in later choice.

3.13 Worked end-to-end example: quadratic optimization plugin

A RAG plugin is complex enough that mistakes can hide behind domain details. A tiny mathematical plugin is useful for testing the campaign semantics independently.

System

Let the parameter be $x \in \mathbb{R}$. Each workload case supplies a target t . Define noisy loss

$$L(x, t, \epsilon) = (x - t)^2 + \epsilon,$$

where $\epsilon \sim \text{Uniform}[-\eta, \eta]$.

The optimization goal is to minimize both loss and absolute distance $|x - t|$.

Parameter space

```
type Spec struct {
  X      float64
  Noise  float64
}
```

A lens focuses on X . The proposer considers candidate values $\{1, 2, 3, 4\}$. Workload targets are $\{2.8, 3.0, 3.2\}$.

Evaluation kernel

For case t_i , repeat r , and seed $s_{i,r}$:

$$K_x(t_i, s_{i,r}) = ((x - t_i)^2 + \epsilon(s_{i,r}), |x - t_i|).$$

Baseline and candidate share the coordinate seed, so noise is paired when the implementation uses the same noise derivation.

Decision policy

1. require all cells to complete;
2. require mean loss not to increase;
3. require mean absolute distance not to increase;
4. choose the candidate with greatest target improvement, subject to gates.

What this example teaches

The campaign engine does not need to know that the domain is quadratic. It sees:

- typed candidate specs;
- intervention IDs;
- workload cases;
- trial coordinates;
- metric vectors;
- decision gates.

If the same engine can run the quadratic plugin and a RAG plugin, we have evidence that the core abstraction is genuinely domain-neutral.

Exercise 3.4

Modify the quadratic example so that larger x also incurs operational cost $0.1x$. Design a decision policy that treats loss noninferiority as a hard gate and cost as a preference. Compare this with minimizing $\text{loss} + \text{lambda} * \text{cost}$.

3.14 Worked RAG example: stochastic answer quality

Assume a retrieval candidate changes vector weight but keeps the index fixed. For each case, baseline and candidate produce evidence sets E_B, E_C . An answer model then samples answers

$$A_B \sim G(\cdot \mid E_B, q, \omega), \quad A_C \sim G(\cdot \mid E_C, q, \omega).$$

A judge kernel evaluates groundedness and completeness:

$$J : (q, E, A) \rightarrow \mathcal{D}(S).$$

The full evaluation is a composition of kernels. The native artifact should retain:

```
query
release/candidate ID
retrieval trace
admitted evidence
```

generation request/response identity
 answer contract result
 judge request/response identity
 metric projection
 usage/cost/latency
 failure/fallback observations

Paired evidence is indexed by exact (case, repeat, arm) coordinates. A candidate that fails generation on one repeat does not get compared on only the successful subset.

Definition 3.21 — Exact pairing relation

Let B and C be sets of trial results indexed by coordinates (i, r) . A comparison is **exactly paired** when

$$\text{dom}(B) = \text{dom}(C) = I \times R.$$

If a coordinate is missing, the comparison is incomplete unless the missingness itself has been materialized as a terminal failure outcome at that coordinate.

Counterexample 3.9 — Pair after dropping failures

Baseline succeeds on cases $\{1, 2, 3, 4\}$, candidate succeeds on $\{1, 2, 4\}$. If comparison intersects successful cases and evaluates $\{1, 2, 4\}$, case 3 vanishes precisely because the candidate failed. The comparison is biased toward success.

3.15 Simpson’s paradox and protected strata

Motivation

An aggregate improvement can hide regression in important subgroups. This is not merely a fairness issue; RAG workloads have natural strata such as access role, source type, query intent, freshness, and language.

Worked example 3.14

Suppose two query classes have the following hit rates:

Class	Baseline	Candidate	Candidate traffic share
exact policy lookup	0.95	0.91	20%
broad discovery	0.60	0.72	80%

The candidate improves aggregate hit rate if discovery dominates, yet policy lookup regresses by four points. If policy lookup is protected, the aggregate winner is not eligible.

Definition 3.22 — Protected stratum

A **protected stratum** is a subset of workload coordinates for which a separate gate or noninferiority margin is required.

Strata should be declared before inspecting candidate results when possible. Otherwise the analysis can become a search for whichever slice makes the desired candidate look favorable.

Exercises 3.5

1. Construct numerical data exhibiting Simpson's paradox where candidate improves within both strata but loses in aggregate because traffic proportions differ.
2. List five RAG strata you would protect in an administrative knowledge system.
3. Explain the difference between a protected stratum and a diagnostic slice discovered after the experiment.

3.16 Campaign provenance and evidence custody

Motivation

When an optimizer becomes adaptive, provenance is not optional. The next proposal may depend on previous evidence. If that evidence cannot be reconstructed, neither can the proposal trajectory.

Definition 3.23 — Evidence graph

An **evidence graph** records immutable relationships such as:

```
baseline spec
-> candidate intervention
-> realized candidate spec
-> build artifacts
-> trial coordinates
-> native trial artifacts
-> metric projections
-> comparison artifact
-> gate results
-> decision
-> next proposal
```

Every node has stable identity; edges have typed roles.

The evidence graph is a retrospective counterpart to the prospective dependency graph of Chapter 2.

Worked example 3.15 — Reproducing an LLM proposal

If an LLM proposer sees a diagnostic report and produces a new prompt patch, retain:

- proposer model identity;
- proposer prompt/policy identity;
- exact evidence projection presented;
- seed or decoding parameters if applicable;
- raw proposal artifact;
- validation result;
- resulting intervention ID.

The campaign can then explain not only whether the candidate won but why it was proposed.

3.17 Chapter synthesis: an optimizer is an experimental state machine

The mathematical picture now has four layers:

1. **System semantics.** Parameterized morphisms and plans describe what the candidate system does.
2. **Stochastic semantics.** Markov kernels describe distributions of trial outcomes.

3. **Evidence semantics.** Exact coordinates, couplings, native artifacts, and metrics describe what was observed.
4. **Decision semantics.** Gates and partial orders describe how evidence changes campaign state.

This separation prevents a common architectural collapse in which “optimizer” means proposer + executor + evaluator + judge + scorer + deployer.

A minimal campaign loop can be written:

```
state = load_or_create_campaign()
while not terminal(state):
    interventions = proposer(state.visible_evidence)
    for I in validate(interventions):
        candidate = realize(state.baseline, I)
        for (case, repeat) in missing_coordinates(candidate):
            seed = coordinate_seed(case, repeat)
            result = evaluator(candidate, case, seed)
            append(result)
        comparison = pair_exactly(baseline, candidate)
        decision = policy(comparison)
        append(decision)
    state = reduce(events)
```

Every line names an interface with semantics developed in the chapter.

Chapter 3 review problems

1. Derive the Kleisli composite of two finite kernels and verify that the resulting probabilities sum to one.
2. Design a coupling for comparing two RAG candidates when query rewriting is stochastic but chunking differs between arms.
3. Give a metric vector and gate sequence where the Pareto-superior candidate is nevertheless ineligible.
4. Define a campaign event reducer and state three invariants you would property-test.
5. Explain how the Blackwell notion of experiment informativeness supports retaining native trial artifacts even when gates use only metric projections.
6. Model a human reviewer as a stochastic kernel. What are the conditioning inputs? What would constitute a useful coupling across candidates?
7. A proposer learns from every completed trial rather than waiting for whole-candidate evaluation. What changes in the campaign coalgebra and in reproducibility requirements?
8. Describe one way to sandbox an LLM proposer so that it can be adaptive without being able to weaken its own evaluation gates.

Chapter 4. A Composable Optimization Architecture for RAG

4.1 Why RAG is a good stress test for the theory

Motivation

Retrieval-augmented generation is not one algorithm. It is a family of systems whose behavior emerges from two broad computations:

1. a **materialization computation** that turns an external corpus into searchable artifacts;
2. a **serving computation** that turns a query and runtime state into evidence, answers, actions, or presentations.

Optimization can intervene on either side. A chunker, representation prompt, embedding model, index backend, fusion weight, reranker, context budget, agent policy, or frontend evidence policy may change. Some changes require expensive rematerialization; others are query-time only. Some preserve ideal semantics while changing approximation or latency. Others intentionally change knowledge projection. Some are safe to evaluate at retrieval level; others require end-to-end sessions.

This is exactly the sort of domain for which a small compositional kernel is useful. RAG is complex enough to need the structure but specific enough that the core must not absorb its vocabulary.

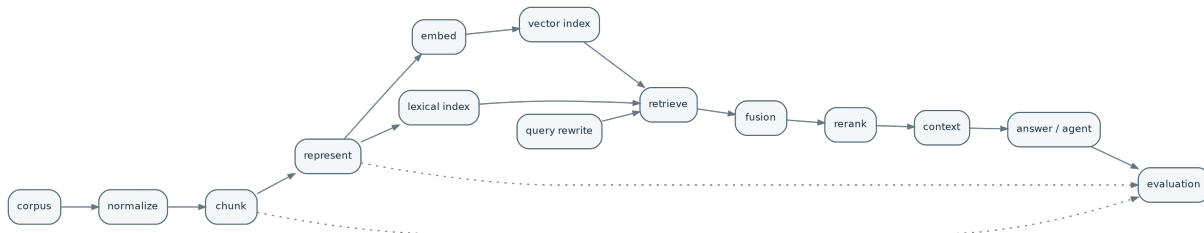


Figure 11: A dependency graph spanning corpus materialization and query-time behavior.

4.2 A denotation for a RAG release

Before defining plugin interfaces, we need to say what a complete RAG configuration *means*.

Definition 4.1 — RAG release

A **RAG release** is an immutable, behavior-complete parameter object that identifies the material and policy inputs required to interpret a query.

A schematic release is

$$R = (S, B, I, Q, A, P),$$

where:

- S is source/corpus snapshot identity;
- B is build specification: normalization, chunking, representations, embeddings;
- I is index material and backend specification;
- Q is query/retrieval policy;
- A is answer or agent policy;
- P is product policy such as authorization, structured facts, and presentation.

A direct-search denotation can be modeled as

$$\llbracket R \rrbracket_{search} : U \otimes Qry \rightarrow \mathcal{D}(Outcome_{search}),$$

where U is subject/runtime context.

An answer denotation is

$$\llbracket R \rrbracket_{answer} : U \otimes Ctx \otimes Qry \rightarrow \mathcal{D}(Outcome_{answer}).$$

An agentic system may be better modeled as a state machine or coalgebra over conversation state.

Why “release” is larger than “index”

Suppose two servers open the same vector and lexical index, but one uses a different synonym set or reranker model. Their behavior differs even though their index artifact is identical. If both call themselves release index-42, optimization evidence cannot identify what was evaluated.

The release parameter object must therefore include every behaviorally material input relevant to the experiment. Runtime details such as worker count can remain execution identity unless they affect observable semantics.

Design callout: a candidate is a new release specification, not a bag of patches.

A patch/intervention explains *how the candidate differs* from baseline. The realized candidate should still be a complete immutable specification whose identity can stand alone.

4.3 RAG indexing as compositional derivation

Motivation

An indexing pipeline often looks linear in documentation:

```
load -> chunk -> embed -> index
```

Real systems contain branches and derived representations:

```
source
-> normalize
-> chunk
    -> raw representation -----> lexical index
    -> contextual representation --\
    -> generated questions -----+--> embeddings -> vector index
    -> summary -----/
```

The free monoidal plan language can express this graph directly. Each stage is supplied by a domain plugin.

Definition 4.2 — Derivation stage

A **derivation stage** is a primitive morphism whose output semantic identity is a function of declared semantic inputs:

$$f : X \rightarrow Y, \quad \text{ID}(y) = H(\text{opID}, \text{ID}(x), \text{staticSpec}, \dots).$$

A deterministic derivation can be content-addressably cached. A stochastic derivation must additionally identify or retain the realized stochastic output and its generating conditions.

Worked example 4.1 — Chunking

```
type Chunker interface {
    Descriptor() OperationDescriptor
    Chunk(context.Context, ChunkSpec, Document) ([]Chunk, error)
}
```

A product need not expose the interface exactly this way. In the compositional signature, the important operation is:

$$\text{chunk} : \text{ChunkSpec} \otimes \text{Document} \rightarrow \text{ChunkSet}.$$

A chunk should retain source lineage:

```
type Chunk struct {
    ID          ChunkID
    DocumentID  DocumentID
    Start, End  int
    TextDigest  Digest
    Text        string
}
```

The chunk ID depends on document revision, chunker identity, and exact span/text semantics. This makes a chunking intervention's invalidation effects mechanically visible.

Worked example 4.2 — Generated representations

A representation generator may be stochastic or provider-backed:

$$\text{represent} : \text{PromptSpec} \otimes \text{Chunk} \circ \mathcal{D}(\text{Representation}).$$

A realized representation is *searchable derived material*, not automatically authoritative evidence. It retains a link to its source chunk.

This distinction prevents an optimization mistake: generated questions can improve retrieval, but the answer should normally cite source evidence, not the generated question as an independent fact.

Definition 4.3 — Derivation dependency

If stage g consumes output of stage f , then semantic changes in f 's output identity propagate to g . The release-specific plan induces a dependency graph.

A build-affecting intervention is therefore evaluated by two functions:

$$\text{apply} : R \times I \rightarrow R'$$

and

$$\text{impact} : R \times I \rightarrow (\text{reuse}, \text{rebuild}, \text{reevaluate}).$$

Worked example 4.3 — Chunking versus fusion

Change chunk size. Reuse source snapshot and normalized documents. Rebuild chunks, affected representations, embeddings, indexes. Reevaluate retrieval and downstream answer/session outcomes.

Change vector fusion weight. Reuse every build artifact. Reexecute fusion and every downstream evaluation whose outcome may change.

A shared optimizer should not need RAG-specific `if chunk_size` changed code. The RAG plugin's dependency declarations and optics give the planner enough information to compute the closure.

4.4 Querying as a typed retrieval algebra

Motivation

A production query path contains operations whose ordering is semantically meaningful:

rewrite → channels → collapse → filter → fuse → rerank → hydrate → admit.

Not every route uses every operation. The core should provide composition; the RAG package should provide the vocabulary and laws.

Definition 4.4 — Candidate

A **retrieval candidate** is a non-authoritative ranked reference produced by a channel. It should contain enough identity and score information to combine or filter results without necessarily hydrating full source text.

```
type Candidate struct {
    EvidenceID EvidenceID
    SourceID   SourceID
    Score      FiniteScore
    Rank       int
    Channel    ChannelID
    Metadata   CandidateMetadata
}
```

Definition 4.5 — Evidence

Evidence is material that has passed the application's admission and authorization rules and may be supplied to an answer or presentation process.

A representation hit becomes evidence only after it resolves to an authoritative source item under the release's evidence policy.

Definition 4.6 — Retrieval plan

A **retrieval plan** is a typed plan assembled from primitives such as:

- query rewrite;
- lexical/vector/structured channel search;
- representation collapse;
- authorization/filtering;
- rank fusion;
- reranking;
- hydration;
- evidence admission.

The plan can be interpreted directly for execution and separately for security, cost, and provenance.

Worked example 4.4 — Hybrid retrieval

Let:

$$L : P_L \otimes Q \rightarrow R_L, \quad V : P_V \otimes Q \rightarrow R_V.$$

A fusion policy

$$F : P_F \otimes (R_L \otimes R_V) \rightarrow R$$

creates a parameterized composite with parameter object

$$P_L \otimes P_V \otimes P_F.$$

If the product exposes a single `RetrievalSpec`, a reparameterization compiles it into these low-level parameter objects.

Worked example 4.5 — Weighted reciprocal-rank fusion

For channel set C , rank $r_c(d)$, rank constant $k > 0$, and channel weight $w_c \geq 0$, define

$$\text{RRF}(d) = \sum_{c \in C} \frac{w_c}{k + r_c(d)}.$$

A fusion plugin should specify deterministic tie-breaking for equal finite scores. One possible total order is:

1. decreasing fused score;
2. increasing best channel rank;
3. stable document/source identity;
4. stable chunk identity.

The exact rule is domain policy; the **requirement that it be total and deterministic** belongs in shared law tests.

Counterexample 4.1 — Hidden hydration inside reranking

If a `rerank(candidates)` callback internally hydrates source text, static policy analysis may see only a `rerank` operation and miss the disclosure. A better primitive descriptor states whether the operation consumes IDs, metadata, or text and declares remote effects. A product security interpreter can then verify authorization dominates disclosure.

4.5 Open ports and plugin interfaces

Motivation

In a compositional architecture, the word **port** should have a precise meaning. It does not mean a TCP port. It is the typed boundary at which an open component expects or exposes information.

A campaign machine may be “open” with respect to proposal strategy, evaluator, artifact store, or decision policy. A RAG plan may be open with respect to a retrieval backend or reranker. Composition closes ports by connecting compatible interfaces.

Definition 4.7 — Port

A **port** is an ordered typed interface boundary:

$$P = (A_1, \dots, A_n).$$

A plan has input port P_{in} and output port P_{out} . Two plans can compose sequentially when the output port of the first matches the input port of the second.

In a software IR:

type Port []SchemaID

The word “open” means that some required port remains supplied by the surrounding environment rather than internally wired.

Worked example 4.6 — Open evaluator

The optimization core can define a campaign over a port

Candidate × Case × Seed -> TrialOutcome

without knowing its implementation. A RAG plugin closes that port with its evaluator; a compiler plugin supplies another evaluator.

This is a cleaner extension point than a plugin receiving a mutable Campaign object.

Worked example 4.7 — Open artifact store

A plan interpreter may require a capability port

ArtifactStore

for content-addressed writes. Local filesystem, S3-compatible storage, and in-memory test stores can implement the same capability. The core cares about the semantic contract—immutable put/get by digest—not the transport.

Side topic: ports versus interfaces. A Go interface is one way to implement an open port. A schema-typed wire in a plan is another. The mathematical notion is about compositional boundary; the language-level interface is an engineering realization.

4.6 The plugin family for a production RAG field

A mature RAG optimization system benefits from several plugin interfaces rather than one giant RAGPlugin.

4.6.1 Corpus/source plugin

Responsibilities:

- capture source snapshots or revisions;
- normalize documents;
- expose source identity and policy metadata;
- retain watermarks or version information where relevant.

Possible operations:

capture : SourceRequest -> CorpusSnapshot

normalize : NormalizeSpec × CorpusSnapshot -> NormalizedCorpus

4.6.2 Chunking plugin

chunk : ChunkSpec × NormalizedCorpus -> ChunkSet

Laws:

- deterministic under claimed semantics;
- every chunk resolves to exactly one source revision/span;
- valid non-overflowing ranges;
- stable total ordering;
- canonical IDs.

4.6.3 Representation plugin

`represent : RepresentationSpec × ChunkSet -> RepresentationSet`

May be deterministic or stochastic. Generated outputs retain source lineage and provider/prompt identity.

4.6.4 Embedding plugin

`embed : EmbeddingSpec × RepresentationSet -> VectorSet`

`embedQuery : EmbeddingSpec × Query -> QueryVector`

A law should ensure build and query embedding specs are compatible. A release must not silently combine vectors from one model with query embeddings from another.

4.6.5 Index backend plugin

Capabilities may include:

```
type Capabilities struct {
  FullBuild      bool
  Upsert         bool
  Delete         bool
  FilterPushdown bool
  SnapshotRead   bool
  Exact          bool
  Compact        bool
}
```

Operations:

`build : IndexSpec × Entries -> IndexArtifact`

`open : IndexArtifact -> SearcherCapability`

`search: SearcherCapability × Query × Filter -> Ranking`

The optimizer can reject a candidate whose required semantics exceed backend capabilities before running an expensive campaign.

4.6.6 Fusion/reranking plugin

Fusion is often pure and deterministic. Reranking may be local or remote, stochastic or deterministic, and may disclose text. Its descriptor should say so.

4.6.7 Evidence/admission plugin

This product-owned layer decides what ranked items may become evidence under subject and release policy.

`authorize : Subject × CandidateSet -> AuthorizedCandidateSet`

`hydrate : AuthorizedCandidateSet -> HydratedEvidence`

`admit : EvidencePolicy × HydratedEvidence -> ContextEvidence`

The ordering is security-sensitive and should be checkable by an interpreter.

4.6.8 Answer/agent plugin

A retrieve-then-generate answer stage can be a stochastic kernel. An agent plugin exposes a bounded state machine with tool-call ports. Product prompts and tool schemas belong to the release parameter object.

4.6.9 Evaluator plugin

A domain evaluator consumes a candidate release and a workload case and returns a **native artifact plus stable metric projection**.

```
type TrialResult struct {
    Status      Status
    Native      ArtifactRef
    Metrics     MetricVector
    Trace       ArtifactRef
    Failure     *Failure
}
```

This preserves domain richness without teaching the campaign core what nDCG, citation support, or widget validity means.

4.7 ragkit and ragopt: separating domain semantics from campaign custody

Motivation

The motivating codebase already suggests a useful architectural split.

- ragkit is the natural owner of reusable RAG functionality and semantics.
- ragopt is the natural owner of optimization-run mechanics: candidates, paired evaluation, resumability, comparison, gates, and reporting.

The mistake would be to put all RAG optimization semantics into ragopt or all experiment mechanics into ragkit.

Definition 4.8 — Domain layer versus experiment layer

The **domain layer** defines:

- legal parameter spaces and interventions;
- build/query operations;
- artifact semantics;
- native evaluators and metric meaning;
- domain laws and capabilities.

The **experiment layer** defines:

- trial coordinates;
- exact pairing;
- seed/coupling policy;
- durable evidence custody;
- comparison mechanics;
- gate execution;
- proposal/campaign state.

A thin adapter binds them.

Proposed package shape

```

optimization-kernel/
  core/          schemas, envelopes, identities, plans
  optic/         lawful interventions
  prob/          seeds, finite/reference kernels, couplings
  experiment/    coordinates, evidence, comparisons
  campaign/      event reducer, resume, proposal loop
  plugin/        signature registry and law harness

ragkit/
  corpus/
  derive/
  index/
  query/
  answer/
  evidence/
  eval/
  plugins/       concrete backend/provider adapters

ragopt/
  candidate/
  runstore/
  compare/
  gate/
  report/
  ragspace/      thin adapter importing ragkit domain types

```

The exact repository boundaries can differ. The important condition is dependency direction: the domain-neutral campaign core does not import RAG packages.

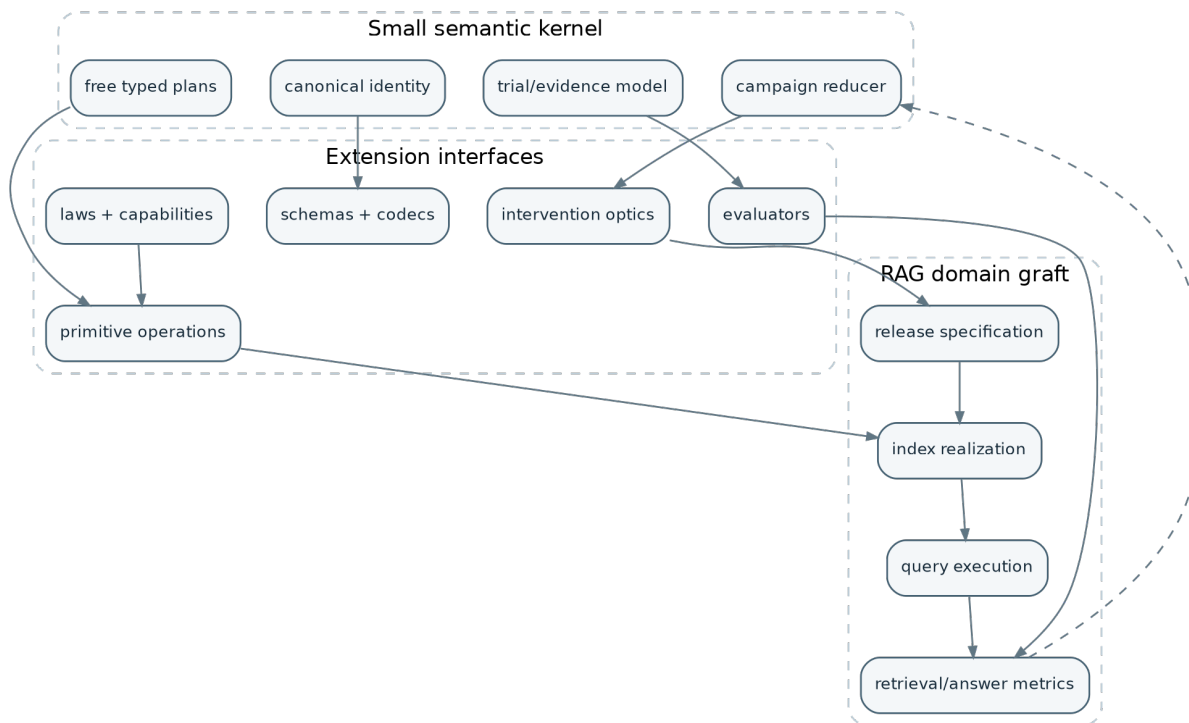


Figure 12: A complete architecture with a small semantic kernel, extension interfaces, and a RAG domain graft.

Counterexample 4.2 — ragopt knows every RAG parameter

If ragopt contains fields such as ChunkSize, EmbeddingModel, RRFK, and RerankTopK, every new RAG mechanism requires changing the supposedly generic optimizer. Compiler or scheduling optimization will not fit. The optimizer has become a second RAG model.

Instead, ragopt should see intervention IDs, candidate/release IDs, dependency summaries, trial coordinates, native artifact references, and metric projections.

4.8 Candidate spaces as plugin interfaces

Motivation

An optimizer needs a way to ask “what may I change?” without knowing the domain’s configuration structure.

Definition 4.9 — Candidate space

A **candidate space** exposes a baseline semantic object and legal interventions.

```
type Space interface {
    ID() string
    Schema() SchemaID
    Baseline(context.Context) (Envelope, error)
    Proposals(context.Context, Envelope) ([]Intervention, error)
    Apply(context.Context, Envelope, Intervention) (Envelope, error)
    Impact(context.Context, Envelope, Intervention) (Impact, error)
}
```

The space is domain-owned. A generic proposer can rank or select interventions, but only the space can validate them.

Worked example 4.8 — A typed RAG search space

```
type RAGSpace struct {
    VectorWeight Lens[ReleaseSpec, float64]
    ChunkSize    Lens[ReleaseSpec, int]
    RerankModel  Lens[ReleaseSpec, ModelRef]
}
```

The space may expose discrete proposals:

```
vector_weight in {0.5, 0.8, 1.0, 1.2, 1.5}
chunk_size in {600, 800, 1000, 1200}
reranker in {none, local-v3, remote-v5}
```

or a continuous/speculative proposal API. The important part is that every proposal becomes a validated intervention with semantic class and impact closure.

Side topic — Conditional spaces

Not every parameter is active in every configuration. efSearch is meaningful only for HNSW. RerankPool may be irrelevant when reranking is disabled. The candidate space is therefore often a dependent or conditional space, not a Euclidean box.

A plugin can expose:

```
type Availability interface {
    Active(spec Envelope, optic OpticID) bool
}
```

or represent backend-specific parameters through sum types. This is another reason to avoid a flat numeric vector as the semantic source of truth.

4.9 Evaluation fidelity is determined by the intervention

Motivation

A fusion-weight change can be evaluated cheaply on frozen channel rankings. A chunking change cannot. An agent prompt change cannot be accepted from retrieval metrics alone. We need a mapping from intervention class/dependency closure to the minimum evaluation fidelity.

Definition 4.10 — Fidelity

An **evaluation fidelity** is a level of experimental realism and cost. A useful RAG ladder is:

1. schema and law validation;
2. deterministic retrieval on a frozen local fixture;
3. exact retrieval evaluation on a labeled suite;
4. repeated answer evaluation;
5. multi-turn/session evaluation;
6. refresh/load/failure simulation;
7. shadow traffic;
8. canary traffic.

An intervention has a **minimum fidelity requirement** determined by what it can change.

Worked example 4.9 — Fusion weight

A fusion-weight sweep can first reuse channel rankings:

```
lexical ranking artifact ----\
                                +--> refusion(w) -> retrieval metrics
vector ranking artifact ----/
```

This experiment validly estimates the effect of fusion under fixed channel outputs. It does not estimate the effect of changing chunking, embeddings, or reranking.

Promising weights can then advance to answer evaluation.

Worked example 4.10 — Chunker

A chunker change starts earlier in the dependency graph. Its retrieval evaluation needs rebuilt artifacts. Its answer evaluation may need source-span labels projected onto candidate chunks because old chunk IDs no longer exist.

Worked example 4.11 — Agent tool description

Changing the model-visible search-tool description may change whether and how often the agent retrieves. Retrieval-only tests that call search directly bypass the intervention. Minimum fidelity is an agent/session evaluation.

Counterexample 4.3 — One benchmark for every parameter

A single benchmark creates false confidence. It may be too expensive for query-weight sweeps and too shallow for agent-policy changes. Dependency-aware fidelity preserves both efficiency and validity.

4.10 Native artifacts and projections

Motivation

A generic campaign core needs comparable fields but a RAG investigator needs detailed traces. We solve this by separating the native trial artifact from its projection.

Definition 4.11 — Native trial artifact

A **native trial artifact** is the domain-specific immutable record sufficient for diagnosis and future remeasurement.

For retrieval:

release ID
case/repeat/arm coordinate
query and filters
channel rankings
collapse/fusion contributions
reranker request/outcome
final evidence ranks
latency/usage
failures/fallbacks

For answer/session evaluation, it may additionally include answer, citations, tool calls, structured facts, and judge artifacts.

Definition 4.12 — Metric projection

A **metric projection** is a deterministic or versioned transformation

$$\pi : \text{NativeArtifact} \rightarrow \text{MetricVector}.$$

The campaign core stores both the native artifact reference and the projection used by a comparison. If the product later adds a new diagnostic metric that can be computed from retained artifacts, it can reproject without rerunning providers.

4.11 Worked campaign A: fusion-weight optimization

We now execute a complete campaign using the abstractions developed throughout the book.

Step 1 — Baseline

A baseline release R_0 contains:

chunker: heading-aware/v3
embedding: model E17
lexical index: artifact L
vector index: artifact V
fusion: k=60, lexicalWeight=1.0, vectorWeight=1.0
reranker: disabled

Step 2 — Space and lens

The RAG plugin exposes a lens

$$L_w : R \rightsquigarrow \mathbb{R}_{\geq 0}$$

focused on vector weight. Proposals are $w \in \{0.6, 0.8, 1.2, 1.4\}$.

Step 3 — Impact closure

The intervention targets `query.fusion.vector_weight`. The dependency graph marks build artifacts reusable:

```
reuse: corpus, chunks, representations, embeddings, lexical/vector indexes
recompute: fused rankings, admitted evidence
reevaluate: retrieval metrics, answer metrics if candidate advances
```

Step 4 — Trial coordinates

For cases $i = 1, \dots, n$ and repeats $r = 1, \dots, R$, the run store requires baseline and candidate results at every coordinate.

For deterministic retrieval, repeats may be unnecessary at the retrieval fidelity; repeated answer trials are introduced later.

Step 5 — Retrieval evidence

The evaluator writes a native ranking artifact and projects:

(Recall@10, MRR, nDCG@10, ScoredCandidates).

Step 6 — Gates

```
coverage == 100%
security violations == 0
Recall@10 lower confidence difference >= -0.005
MRR lower confidence difference > 0
nDCG protected strata noninferior
Eligible candidates move to answer fidelity.
```

Step 7 — Answer fidelity

Baseline and candidate use paired case/repeat seeds. Generation and judge outputs are retained. Gates add groundedness, answer completeness, provider calls, latency, and cost.

Step 8 — Decision

If multiple candidates survive, choose from their Pareto frontier according to product preference. The campaign emits a promotion report referencing the complete candidate release spec and evidence graph. It does not mutate production directly.

What the core knew

The core knew:

- a schema-identified baseline;
- intervention IDs;
- trial coordinates;
- artifacts;
- metrics and directions;
- gate results;
- campaign events.

It did not know what RRF or nDCG meant.

4.12 Worked campaign B: chunking optimization

This example shows why materialization and query optimization must share one field.

Step 1 — Intervention

Change chunk target from 1,200 to 800 units with the same overlap policy.

Semantic classes: knowledge, relevance.

Step 2 — Impact

The planner computes:

```
reuse:
  source snapshot
  normalized documents

rebuild:
  chunk records
  generated representations for affected chunks
  embeddings for changed representations
  lexical index
  vector index

reevaluate:
  retrieval labels projected from source spans
  retrieval metrics
  answer/session metrics
```

Step 3 — Materialization

Every deterministic stage uses content-addressed keys. Unchanged chunks or representations, if their semantic IDs truly survive, can be reused. Stochastic generated representations require careful policy: reuse the retained old output only when its semantic key remains identical; otherwise the candidate receives a new realized artifact.

Step 4 — Label projection

If the benchmark labels relevant **documents**, old labels survive trivially. If it labels old chunk IDs, the benchmark is coupled to the baseline chunker. A stronger dataset labels authoritative source spans or propositions and provides a versioned projection to candidate chunks.

This illustrates a deep point: the evaluation schema itself can determine which interventions are scientifically comparable.

Step 5 — Refresh amplification metric

Define

$$A_{refresh} = \frac{\text{number of derived items recomputed}}{\text{number of source documents changed}}.$$

A chunker that improves static retrieval but causes extreme invalidation under common edits may be poor for a frequently changing production corpus. Optimization should include both quality and maintenance cost when production dynamics matter.

Step 6 — Decision

Security and lineage laws first; retrieval noninferiority and target gains next; answer quality next; build cost, index size, and refresh amplification after semantic gates.

4.13 Worked campaign C: exact versus ANN index

Motivation

An approximate vector backend is an ideal example of an intervention whose semantics are not exact equality.

Let $E(q)$ be the exact nearest-neighbor ranking and $A_\theta(q)$ the ANN ranking under parameter θ .

A candidate claim may be:

$$\mathbb{P}_{q \sim W} [\text{Recall@k}(A_\theta(q), E(q)) \geq \rho] \geq 1 - \alpha.$$

The exact backend is an **oracle** for approximation quality.

Candidate parameters

- backend implementation/version;
- graph construction parameters;
- efSearch;
- quantization;
- partition count;
- filter strategy.

Required evidence

- exact-oracle recall at several k ;
- protected filters/source strata;
- deterministic or distributional reproducibility class;
- p50/p95/p99 latency;
- memory and index size;
- build time;
- update/delete behavior if production uses incremental maintenance;
- downstream answer noninferiority.

Why the intervention optic matters

efSearch is a query-time focus. M and efConstruction are build-time focuses. A backend change from exact SQLite to HNSW is a larger prism-like variant switch. These interventions have different impact closures despite sharing one conceptual “vector index” component.

4.14 Optimization and production release semantics

Motivation

The optimizer produces evidence about candidates, but production queries need stable release identities. Promotion should connect the two worlds without letting the optimizer become deployment authority.

Definition 4.13 — Promotion artifact

A **promotion artifact** is an immutable record containing:

- baseline release ID
- candidate release ID
- intervention ID
- campaign ID
- comparison artifact
- ordered gate results
- protected-stratum evidence
- operational budget evidence
- human/automation decision identity

An external activation service can require a valid promotion artifact before changing the active release.

Why activation is outside the optimizer

Separating promotion from activation prevents an optimizer from changing the environment it is measuring without a product-controlled boundary. It also supports staged, canary, or human-reviewed rollout.

The categorical view is that optimization produces a morphism in an **evidence/decision system**; production activation is a separate effectful interpreter with stronger authority.

4.15 Plugin design rules for a small but strong backbone

We can now state concrete rules.

Rule 1 — The core owns composition, not domain vocabulary

The core knows schemas, operations, plans, optics, effects, artifacts, evidence, and campaigns. It does not know chunks or compilers.

Rule 2 — Plugins expose generators, not arbitrary engine callbacks

Every executable domain primitive has a descriptor visible before execution.

Rule 3 — Plugins own semantic validation

The RAG plugin decides whether a chunking spec is legal and whether a benchmark target granularity is compatible. The core checks that validation happened and records the result.

Rule 4 — Effects are declared

A remote reranker cannot masquerade as a pure ranking function. Network/text-disclosure capabilities are visible to plan analysis.

Rule 5 — Interventions are lawful and typed

Use lenses/optics or explicit multi-field transformations with laws. Do not treat arbitrary mutation as a candidate.

Rule 6 — Native evidence is retained

Metric projection is not the only record of what happened.

Rule 7 — Failure is evidence

Every scheduled coordinate becomes a terminal outcome or remains explicitly incomplete. Failed cells are not filtered before pairing.

Rule 8 — Decision policy is versioned and separate

Changing promotion thresholds does not mutate the underlying trial evidence.

Rule 9 — Optimizers are replaceable controllers

Grid search, Bayesian optimization, evolutionary algorithms, humans, and LLM proposers operate through the same candidate/evidence interfaces.

Rule 10 — Every semantic claim should point to a law or test

“Deterministic,” “cacheable,” “authorization-safe,” “approximation-equivalent,” and “local intervention” are claims with executable obligations.

4.16 A self-contained Go skeleton

The following API is intentionally compact. It is not intended as a final production library; it demonstrates how the mathematics compresses into ordinary interfaces.

Core schemas and operations

```

type SchemaID string
type OperationID string
type Digest string

type Envelope struct {
    Schema SchemaID
    Data []byte
    Digest Digest
}

type Port []SchemaID

type OperationDescriptor struct {
    ID          OperationID
    Inputs      Port
    Outputs     Port
    Effects     []Effect
    Deterministic bool
    Cacheable   bool
    Dependencies []string
}

type Operation interface {
    Descriptor() OperationDescriptor
    Execute(context.Context, []Envelope) Execution
}

```


Plans

```

type Plan struct {
    Kind          PlanKind
    In, Out       Port
    Operation     OperationID
    Children      []*Plan
    Permutation   []int
}

func Seq(ps ...*Plan) (*Plan, error)
func Tensor(ps ...*Plan) (*Plan, error)
func Copy(schema SchemaID) *Plan
func Drop(schema SchemaID) *Plan
func Fold[R any](p *Plan, sig Signature, a Algebra[R]) (R, error)

```

Optics/interventions

```

type Lens[S,A any] struct {
    ID      string
    Get     func(S) A
    Put     func(S, A) (S, error)
}

type Intervention struct {
    ID          string
    Optic       string
    Value       Envelope
    Classes     []SemanticClass
    Targets     []string
    Claims      []Claim
}

```

Evaluation

```

type Coordinate struct {
    Case      string
    Repeat    int
    Arm       string
    Candidate Digest
}

type TrialResult struct {
    Coordinate Coordinate
    Status      Status
    Metrics     MetricVector
    Native      ArtifactRef
    Trace       ArtifactRef
    Failure     *Failure
}

```

Campaign

```

type Space interface {
    Baseline(context.Context) (Envelope, error)
    Proposals(context.Context, Envelope) ([]Intervention, error)
    Apply(context.Context, Envelope, Intervention) (Envelope, error)
}

```

```
    Impact(context.Context, Envelope, Intervention) (Impact, error)
}
```

```
type Runner interface {
    Run(context.Context, TrialRequest) TrialResult
}
```

```
type Policy interface {
    Decide(Comparison) Decision
}
```

This is enough to implement the quadratic example and a nontrivial RAG plugin without adding RAG conditionals to the campaign engine.

4.17 Interpreters for a RAG plan

Consider one release plan. The architecture can compile it into several meanings.

Execution interpreter

Uses local implementations, artifact stores, provider clients, and concurrency.

Dependency interpreter

Returns semantic nodes and computes intervention closure.

Cost interpreter

Estimates build calls, embedding tokens, provider dollars, storage, critical path, and memory.

Disclosure interpreter

Tracks data classification through the plan and verifies remote operations receive only permitted data.

Provenance interpreter

Constructs the prospective lineage graph and binds material artifact references after execution.

Evaluation-fidelity interpreter

Computes the highest minimum fidelity required by changed nodes.

Documentation interpreter

Renders the exact plan into a graph for reports and reviews.

This is the concrete payoff of freeness: a new plugin operation teaches each interpreter its primitive meaning through a descriptor or handler, but the composition logic remains shared.

4.18 Verification strategy

Motivation

A mathematically inspired architecture is useful only if the laws become tests.

We divide verification into layers.

4.18.1 Core algebra tests

- sequence associativity after plan normalization;
- tensor associativity/coherence under canonical representation;
- identity elimination;
- symmetry/permutation validity;
- schema/port typing;
- stable plan identity.

4.18.2 Optic tests

- Get-Put;
- Put-Get;
- Put-Put;
- composition preserves laws over generated valid states.

4.18.3 Plugin conformance

- schema codec round-trip;
- operation output schema;
- deterministic claims;
- cache-key sensitivity;
- capability contracts;
- total ranking;
- source/evidence lineage.

4.18.4 Experiment tests

- deterministic coordinate seeding;
- exact baseline/candidate pairing;
- failure retention;
- resume equivalence;
- idempotent event replay;
- gate order and fail-closed behavior.

4.18.5 RAG-specific properties

- authorization before remote text disclosure;
- build/query embedding compatibility;
- generated representation resolves to source evidence;
- incremental/full-build equivalence when incremental maintenance exists;
- ANN recall against exact oracle;
- citation IDs belong to admitted evidence;
- no mixed release IDs in one pinned trial.

Worked example 4.12 — Property test for total ranking

Generate finite candidates with arbitrary valid finite scores, including ties. A comparator $<$ must satisfy:

Irreflexivity: never $a < a$.

Transitivity: if $a < b$ and $b < c$, then $a < c$.

Total comparability: for distinct canonical identities, exactly one order holds after score/tie rules.

Reject NaN and infinity at the score-construction boundary; otherwise ordinary floating-point comparison does not define the desired total order.

4.19 When the category-theoretic architecture is overkill

A good textbook must state where its own abstraction should not be used.

Counterexample 4.4 — One pure scalar function

If your optimization problem is literally

$$\min_{x \in [0,1]} f(x)$$

where f is deterministic, cheap, pure, and has no artifact or safety semantics, a numerical optimization library is sufficient. Building a free monoidal plan and plugin registry adds ceremony without information.

Counterexample 4.5 — One application with no extension boundary

A small internal script may never need plugins. It can still benefit from explicit trial coordinates and failure preservation, but an algebraic signature registry can be unnecessary.

Counterexample 4.6 — False abstraction before two domains exist

Do not extract `UniversalIndexOptimizer` merely because RAG-TTC and GEC both have an `Optimize` command. Extract shared semantics only where their laws and lifecycle genuinely coincide. Domain-specific code is often healthier than a premature “generic” interface.

The criterion is **semantic compression**: an abstraction is valuable when one small set of laws explains several real implementations and makes new checks possible.

4.20 Migration path from pragmatic RAG code

A mature system should not be rewritten all at once. The categorical kernel can be introduced by progressively making implicit semantics explicit.

Stage 0 — Behavioral fixtures

Capture current retrieval rankings, bundle identities, answer contracts, failure paths, and experiment pairing behavior. These fixtures define what must remain equivalent and what will intentionally change.

Stage 1 — Semantic IDs and native artifacts

Make candidate specs, evaluator versions, and trial artifacts content-addressed or otherwise immutable. Do this before advanced plan syntax.

Stage 2 — Lawful intervention APIs

Replace string-path mutation with typed domain transformations or lenses. Keep legacy configuration serialization as an adapter.

Stage 3 — Operation descriptors

Wrap existing functions with descriptors declaring ports, effects, dependencies, and determinism. Execution still calls the existing code.

Stage 4 — Free plans for one path

Represent one high-value pipeline—perhaps retrieval evaluation—as free syntax. Implement execution and dependency interpreters. Compare against legacy behavior.

Stage 5 — Campaign custody

Move exact trial pairing, resumability, failure retention, and gate reports into the generic experiment kernel while product evaluators remain native.

Stage 6 — Additional interpreters

Add cost, disclosure, provenance, documentation, and fidelity analysis as concrete needs arise.

Stage 7 — Pluggable proposers

Only after evidence and legality are stable should increasingly autonomous proposers, including LLM-based self-optimization, be given control over candidate proposals.

This ordering matters. Autonomous search amplifies whatever semantics already exist; it does not repair weak experimental foundations.

4.21 Capstone worked example: optimizing an entire RAG release

We finish with a campaign containing three intervention families.

Baseline release

Corpus snapshot: C42
 Chunker: MarkdownHeading(size=1000, overlap=100)
 Representations: raw + breadcrumb
 Embedding: E5
 Lexical backend: Bleve profile B3
 Vector backend: exact SQLite
 Query: lexicalK=50, vectorK=50, RRF k=60, vectorWeight=1.0
 Reranker: local R2, pool=20
 Context: maxEvidence=8
 Answer: model A9, grounded contract G4

Intervention family A — Fusion

vectorWeight in {0.7, 0.9, 1.1, 1.3}
 Impact: query/evaluation only.
 Minimum fidelity: retrieval; finalists answer.

Intervention family B — Chunking

size in {700, 850, 1150}

Impact: chunks through all downstream artifacts.

Minimum fidelity: rebuilt retrieval + answer; add refresh-cost metrics.

Intervention family C — ANN backend

backend = HNSW(M=16, efConstruction=200)

efSearch in {40, 80, 120}

Impact: vector index build and vector query.

Minimum fidelity: exact-oracle ANN certification, load, retrieval, answer.

Campaign strategy

A domain-aware proposer can stage the search rather than taking a full Cartesian product.

Phase 1: optimize fusion on current artifacts.

Phase 2: test chunkers using best eligible query policy from phase 1.

Phase 3: certify ANN against exact index for the best release family.

Phase 4: jointly recheck finalists because interactions may exist.

Phase 5: shadow/canary outside the offline campaign kernel.

This strategy is not mathematically guaranteed to find the global optimum because interventions interact. Its advantage is experimental tractability. The field makes the approximation explicit: each phase conditions on a selected incumbent and the final joint recheck tests important interactions.

Evidence graph

R0

```
|-- I_fusion_1 -> R1 -> retrieval trials -> gates
```

```
|-- I_fusion_2 -> R2 -> retrieval trials -> gates
```

```
...
```

R_best_fusion

```
|-- I_chunk_700 -> R3 -> build artifacts -> retrieval/answer trials
```

```
|-- I_chunk_850 -> R4 -> build artifacts -> retrieval/answer trials
```

```
...
```

R_best_quality

```
|-- I_HNSW_40 -> R5 -> ANN oracle + load + answer
```

```
|-- I_HNSW_80 -> R6 -> ANN oracle + load + answer
```

Every arrow is typed. Every candidate has a complete release identity. Every evaluation links to exact artifacts. The generic campaign engine can traverse this graph without understanding the domain fields.

Promotion policy

1. all law and build-integrity checks pass;
2. zero authorization/disclosure violations;
3. no missing paired cells;
4. protected retrieval strata noninferior;
5. grounded answer quality noninferior;
6. target quality improves materially;
7. ANN recall satisfies oracle margin where applicable;
8. p95 latency/cost within budget;
9. candidate is on the eligible Pareto frontier;
10. promotion artifact is handed to external activation.

This is what “optimization” looks like after the semantic structure is made explicit. The loop is still there, but it is no longer carrying the entire theory implicitly.

4.22 Chapter synthesis

The architecture developed in the book can be summarized in one sentence:

Represent domain computation as free typed composition, represent controlled change with lawful optics over parameterized systems, represent uncertain evaluation with explicitly coupled kernels, and represent optimization itself as an evidence-preserving feedback machine whose domain semantics enter only through plugins.

That sentence unifies the mathematics with the software design.

The category supplies **composition**.

The free construction supplies **syntax before semantics**.

Para supplies **compositional parameter spaces**.

Optics supply **local interventions and laws**.

Markov kernels supply **stochastic evaluation**.

Couplings supply **paired comparison semantics**.

Metric vectors and decision algebras supply **constraint-aware selection**.

Coalgebra supplies **iterative behavior and resumability**.

Plugin signatures supply **domain extensibility without callback anarchy**.

RAG supplies a demanding application in which every one of these abstractions earns its place.

Chapter 4 review problems

1. Design a RAG release type whose semantic identity changes when a synonym asset changes but not when worker count changes. State your identity rules precisely.
2. A reranker plugin takes hydrated text and calls a remote endpoint. Define its operation descriptor and a static disclosure rule that a plan validator can check.
3. Draw the invalidation closure for changing only the representation-generation prompt.
4. Design an optic for switching from exact vector search to an HNSW variant. Why is an ordinary scalar lens insufficient?
5. Define native and projected artifacts for a multi-turn RAG session evaluation.
6. Give a minimum-fidelity function for the semantic classes defined in Chapter 2.
7. Propose a plugin-law suite for a chunker that claims stable identities under document-local edits. What exact stability can it honestly promise?
8. Explain how a Bayesian optimizer can be integrated as a proposer without receiving permission to redefine the evaluation metric after seeing candidate results.
9. Suppose two plugins both declare a schema named `Result/v1`. Design a namespace/version rule that prevents accidental type equality.
10. Build a plan interpreter that computes (`totalWork`, `criticalPath`, `remoteCalls`). State the sequence and tensor operations on this carrier.

Appendix A. Selected exercise solutions and hints

This appendix gives solution sketches for representative exercises. It is intentionally incomplete: many exercises are design problems with several defensible answers.

A.1 Category and composition

Chapter 1, Exercise 1.1.1. If $f : A \rightarrow B$ and $g : B \rightarrow C$, then $g \circ f : A \rightarrow C$. Similarly $h \circ g : B \rightarrow D$, and $h \circ g \circ f : A \rightarrow D$. Associativity makes the three-term composite independent of parenthesization.

Chapter 1, Exercise 1.2.3. For stochastic sample, copying after one run yields perfectly correlated outputs. Tensoring two runs yields independent outputs if the tensor interpreter uses product distributions. This is exactly why copy is structural and not syntactic duplication of a box.

Chapter 1 review problem 2. A canonical tensor normalization can flatten nested tensors to an ordered list after resolving explicit symmetries. The normalization should preserve wire order and primitive identities. It need not preserve scheduling metadata if scheduling is execution identity rather than semantic identity.

A.2 Lens laws

For the first projection lens on pairs:

$$\text{get}(a, b) = a, \quad \text{put}((a, b), a') = (a', b).$$

Then

$$\text{put}((a, b), \text{get}(a, b)) = (a, b),$$

$$\text{get}(\text{put}((a, b), a')) = a',$$

and

$$\text{put}(\text{put}((a, b), a'), a'') = (a'', b) = \text{put}((a, b), a'').$$

A chunk-size update with an overlap constraint is more subtle. If `put(size)` can invalidate the existing overlap, the focus is not independent over the entire state space. One solution is to make the focused value a valid pair `(size, overlap)`; another is a partial optic that rejects values incompatible with current residual state. What is *not* valid is silently changing overlap and still claiming a lens focused only on size.

A.3 Couplings

Two fair-coin marginals have many couplings. Independent coupling assigns $1/4$ to each pair. Perfectly correlated coupling assigns $1/2$ to (H,H) and (T,T). Perfect anticorrelation assigns $1/2$ to (H,T) and (T,H). The marginal distributions alone do not identify the joint experiment.

For prompt comparison, shared model seed is useful if the provider interprets seeds consistently across prompts and the semantic goal is common-random-number variance reduction. The case remains the primary paired coordinate; hidden provider nondeterminism must still be acknowledged.

A.4 Cost interpreter

Let a cost carrier be

$$R = \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0} \times \mathbb{N}$$

with components (work, criticalPath, remoteCalls).

For sequence:

$$(w_1, c_1, r_1) ; (w_2, c_2, r_2) = (w_1 + w_2, c_1 + c_2, r_1 + r_2).$$

For tensor, assuming unlimited parallel resources:

$$(w_1, c_1, r_1) \otimes (w_2, c_2, r_2) = (w_1 + w_2, \max(c_1, c_2), r_1 + r_2).$$

The assumptions matter. Under a single shared rate limiter, critical path may not be max; a richer interpreter would track resource classes.

Appendix B. Reference API sketch

The following consolidated API is intended as a design aid, not a drop-in library.

B.1 Core

```
package core
```

```
type SchemaID string
type OperationID string
type Digest string
type Effect string
```

```
type Envelope struct {
    Schema SchemaID
    Data []byte
    Digest Digest
}
```

```
type Port []SchemaID
```

```
type OperationDescriptor struct {
    ID          OperationID
    Version     string
    Plugin      string
    Inputs      Port
    Outputs     Port
    Effects     []Effect
    Deterministic bool
    Cacheable   bool
    Dependencies []string
    Cost        CostHint
}
```

B.2 Plugin registry

```
package plugin
```

```
type Plugin interface {
    Manifest() Manifest
    Install(*Builder) error
    Laws() []Law
}
```

```
type Builder interface {
    RegisterSchema(core.Schema) error
}
```

```

    RegisterOperation(Operation) error
    RegisterOptic(OpticDescriptor) error
    RegisterEvaluator(EvaluatorDescriptor) error
}

```

B.3 Free plans

```
package plan
```

```
type Kind uint8
```

```

const (
    Identity Kind = iota
    Primitive
    Sequence
    Tensor
    Permute
    Copy
    Drop
)

```

```

type Plan struct {
    Kind          Kind
    In, Out       core.Port
    Operation     core.OperationID
    Children      []*Plan
    Permutation   []int
}

```

```

type Algebra[R any] interface {
    Identity(core.Port) (R, error)
    Primitive(core.OperationDescriptor) (R, error)
    Sequence(core.Port, core.Port, []R) (R, error)
    Tensor(core.Port, core.Port, []R) (R, error)
    Permute(core.Port, core.Port, []int) (R, error)
    Copy(core.SchemaID) (R, error)
    Drop(core.SchemaID) (R, error)
}

```

B.4 Optics

```
package optic
```

```

type Lens[S,A any] struct {
    ID      string
    Get     func(S) A
    Put     func(S, A) (S, error)
    Equals  func(S, S) bool
    EqualA  func(A, A) bool
}

```

```

func Compose[S,A,B any](Lens[S,A], Lens[A,B]) Lens[S,B]
func CheckLaws[S,A any](Lens[S,A], []S, []A) error

```

B.5 Probability and seeds

```
package prob
```

```
type Seed [32]byte
```

```
func (Seed) Split(label string) Seed
```

```
type Dist[T comparable] map[T]float64
```

```
type Kernel[A,B comparable] func(A) Dist[B]
```

```
func Compose[A,B,C comparable](Kernel[A,B], Kernel[B,C]) Kernel[A,C]
```

```
func Tensor[A,B,C,D comparable](Kernel[A,B], Kernel[C,D]) Kernel[Pair[A,C],Pair[B,D]]
```

The finite-distribution implementation is a **reference semantics**, not a replacement for continuous or measure-theoretic probability in production.

B.6 Experiment

```
package experiment
```

```
type Intervention struct {
    ID          string
    Optic       string
    Value       core.Envelope
    Classes     []SemanticClass
    Targets     []string
    Closure     []string
    Hypothesis  string
}
```

```
type Coordinate struct {
    Candidate core.Digest
    Case      string
    Repeat    int
    Arm       string
}
```

```
type TrialResult struct {
    Coordinate Coordinate
    Status     core.Status
    Metrics     metric.Vector
    Native      *artifact.Ref
    Trace       *artifact.Ref
    Failure     *core.Failure
}
```

B.7 Campaign

```
package campaign
```

```
type Engine struct {
    Config    Config
    Space     experiment.Space
    Proposer  experiment.Proposer
    Workload  experiment.Suite
    Runner    experiment.Runner
}
```

```
    Policy    decision.Policy
    Store     EventStore
}
```

```
func (e Engine) Run(context.Context) (Result, error)
```

The event store is append-only; the engine recomputes state by reducing events and schedules only missing coordinates.

Appendix C. Mathematical perspective and further reading

The architecture in this book is a synthesis rather than a direct transcription of one categorical framework. Several bodies of work provide the underlying structures.

C.1 Parameterized maps and learning

Fong, Spivak, and Tuyéras show that backpropagation can be understood functorially from parameterized functions to learners, demonstrating that supervised-learning structure can preserve composition [Fong, Spivak, and Tuyéras, 2019]. Cruttwell, Gavranović, Ghani, Wilson, and Zanasi develop categorical foundations of gradient-based learning using lenses, parametrized maps, and reverse derivative categories, and later formulate deep learning in terms of parametric lenses [Cruttwell et al., 2022; 2024].

The optimization architecture here adopts the *parameterized composition* and *lawful bidirectional focus* ideas but deliberately does not require differentiability. RAG interventions are commonly discrete, structural, or effectful.

C.2 Optics

Riley gives a general categorical treatment of optics, showing how lenses, prisms, traversals, and related bidirectional structures fit one construction and developing a notion of optic lawfulness [Riley, 2018]. The implementation in this book starts with ordinary lenses because they cover many release-spec interventions, while leaving the extension boundary optic-shaped.

C.3 Markov categories

Fritz develops Markov categories as a synthetic categorical framework for probability and statistics, including conditional independence and sufficient statistics [Fritz, 2020]. Fritz and Liang study free $\mathbf{gs}$ -monoidal and free Markov categories, connecting string diagrams with combinatorial representations useful for implementations [Fritz and Liang, 2023]. Fritz, Gonda, Perrone, and Rischel study representable Markov categories and comparison of statistical experiments [Fritz et al., 2020].

This book uses finite kernels as executable reference examples, while the conceptual design is meant to remain compatible with richer stochastic semantics.

C.4 Cybernetics and feedback

Capucci, Gavranović, Hedges, and Rischel propose categorical foundations for cybernetic systems interacting bidirectionally with environments and controllers, including open learners [Capucci et al., 2021]. The campaign-as-feedback view in Chapter 3 follows the same compositional motivation while using an explicit event-sourced state machine as the practical implementation boundary.

C.5 Coalgebra

Coalgebra is the standard categorical dual perspective for state-based and potentially infinite behavior. For software readers, the key use in this book is conceptual: optimization campaigns and agentic RAG are better modeled as systems that expose observations and next states than as one pure terminating function.

C.6 Free syntax and interpreters

The free-plan design is a familiar pattern across category theory, typed DSLs, algebraic effects, and tagless-final programming. The universal property explains why a single syntax can be folded into execution, static analysis, cost, provenance, or documentation without duplicating orchestration structure.

Appendix D. Bibliography

- Capucci, Matteo, Bruno Gavranović, Jules Hedges, and Eigil Fjeldgren Rischel. 2021. “Towards Foundations of Categorical Cybernetics.” arXiv:2105.06332.
- Cruttwell, G. S. H., Bruno Gavranović, Neil Ghani, Paul Wilson, and Fabio Zanasi. 2022. “Categorical Foundations of Gradient-Based Learning.” *European Symposium on Programming / associated publication*, arXiv:2103.01931.
- Cruttwell, G. S. H., Bruno Gavranović, Neil Ghani, Paul Wilson, and Fabio Zanasi. 2024. “Deep Learning with Parametric Lenses.” arXiv:2404.00408.
- Fong, Brendan, David I. Spivak, and Rémy Tuyéras. 2019. “Backprop as Functor: A Compositional Perspective on Supervised Learning.” In *Proceedings of LICS 2019*. arXiv:1711.10455.
- Fritz, Tobias. 2020. “A Synthetic Approach to Markov Kernels, Conditional Independence and Theorems on Sufficient Statistics.” *Advances in Mathematics* 370: 107239. arXiv:1908.07021.
- Fritz, Tobias, Tomáš Gonda, Paolo Perrone, and Eigil Fjeldgren Rischel. 2020. “Representable Markov Categories and Comparison of Statistical Experiments in Categorical Probability.” arXiv:2010.07416.
- Fritz, Tobias, and Wendong Liang. 2023. “Free gs-Monoidal Categories and Free Markov Categories.” *Applied Categorical Structures* 31 (2): Article 21. arXiv:2204.02284.
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, et al. 2020. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *Advances in Neural Information Processing Systems* 33.
- Moggi, Eugenio. 1991. “Notions of Computation and Monads.” *Information and Computation* 93 (1): 55–92.
- Riley, Mitchell. 2018. “Categories of Optics.” arXiv:1809.00738.
- Rutten, J. J. M. M. 2000. “Universal Coalgebra: A Theory of Systems.” *Theoretical Computer Science* 249 (1): 3–80.

Closing perspective

A strong optimization architecture is not primarily a collection of search algorithms. It is a language in which we can state what a candidate is, what it changes, what must be recomputed, what randomness is shared, what evidence was observed, what failures count, and what decision rule is allowed to promote it.

Category theory contributes a disciplined answer to composition. Probability theory contributes a disciplined answer to uncertainty. Optics contribute a disciplined answer to local intervention. Coalgebra contributes a disciplined answer to iteration. Plugin interfaces turn these mathematical boundaries into software boundaries.

The practical outcome is modest in size but strong in meaning: a small core that knows how to compose and preserve evidence, surrounded by domain plugins that know what their worlds mean.